

Chapter 6

Signal (Data)

A **signal** is a function that carries or represents **information**.

It shows how a physical quantity changes with respect to another variable such as **time, space, or position**.

For example, a signal can represent:

- **Sound intensity** changing over time
- **Brightness** in an image
- **Temperature** measured by a sensor
- **Stock market prices** changing with time

So, a signal basically describes **how a value changes**.

Types of Signals

1. Continuous Signal

A **continuous signal** exists for **every value** of the independent variable (usually time).

- It is smooth and uninterrupted.
- It can take values at **any moment**.

Examples

- A **sine wave**
- **Temperature variation over time**
- Natural sound waves

2. Discrete Signal

A **discrete signal** exists only at **specific points** instead of every point.

- Usually created by **sampling a continuous signal**.
- Values are available only at certain intervals.

Example

- **Digital audio**, where sound is stored as a series of amplitude samples.

Signals Based on Dimensions

1. One-Dimensional (1D) Signal

A **1D signal** depends on **only one independent variable** (usually time).

Example

- Audio signals
- ECG signals

These signals change only with **time**.

2. Multi-Dimensional Signal

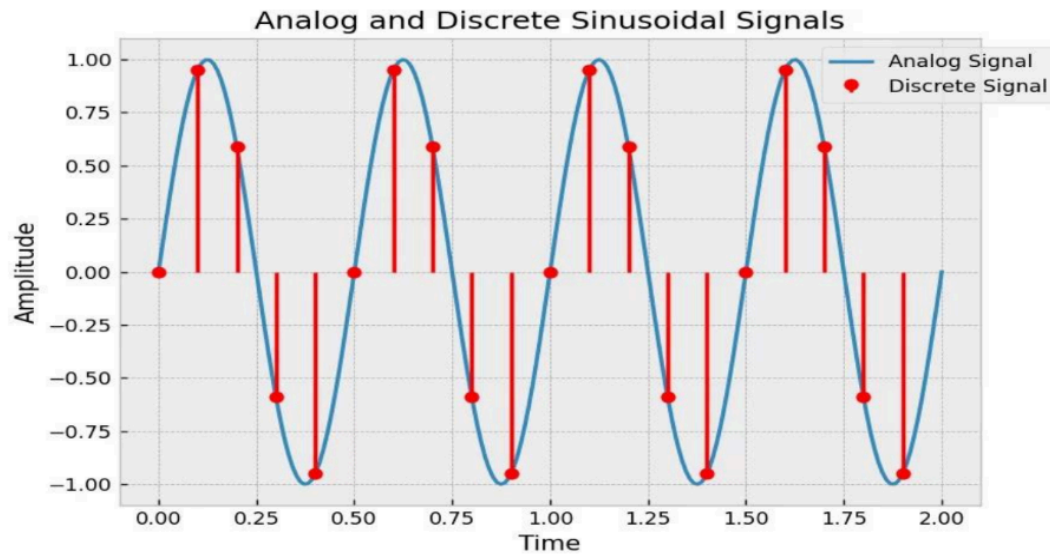
A **multi-dimensional signal** depends on **two or more independent variables**.

Example

- Images depend on **x and y coordinates**
- Videos depend on **x, y, and time**

For instance, in an image, each pixel represents **brightness at a particular location**.

CONTINUOUS (ANALOG) VS DISCRETE SIGNALS



Domain

A **domain** is the **independent variable or dimension** where a signal is represented or analyzed. In simple words, it tells **where the signal exists or how it is measured**.

A signal is described by:

1. Its **values (amplitude)**
2. The **domain** in which it is defined

Common Types of Domains

1. Time Domain / Spatial Domain

This is the **natural way to represent signals**.

- **Time domain:** Shows how a signal changes with time.
- **Spatial domain:** Shows how a signal changes with position in space.

Examples

- An **audio waveform** shows amplitude vs **time**.
- An **image** shows brightness vs **spatial coordinates (x, y)**.

This domain helps us understand **when or where something happens**.

2. Frequency Domain

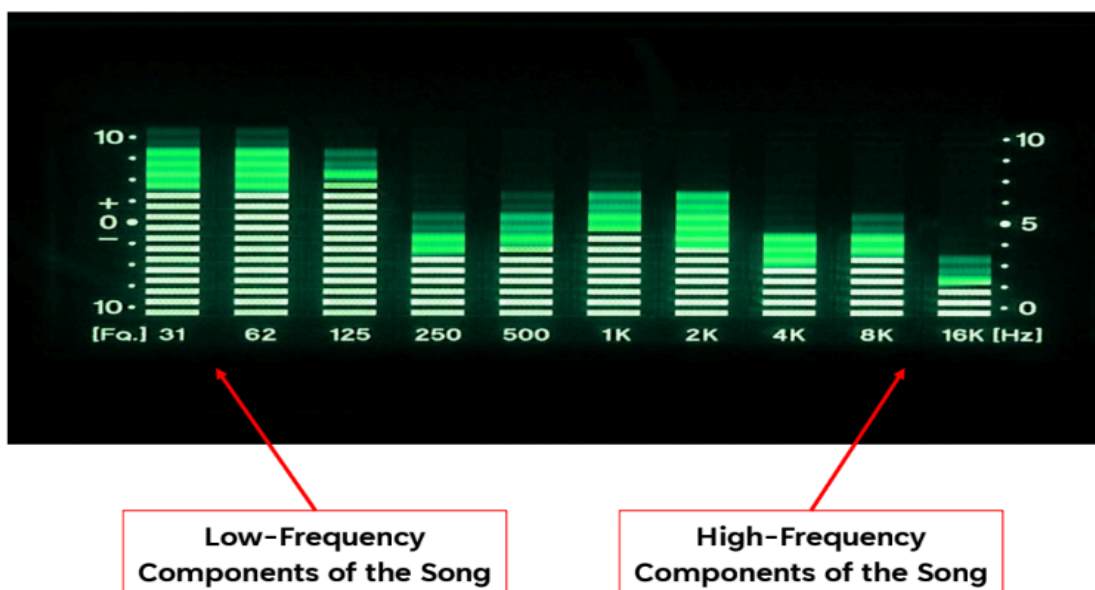
The **frequency domain** represents a signal based on its **frequency components** instead of time or space. This means it shows **what frequencies exist in the signal**. To convert a signal to frequency domain, we use the **Fourier Transform**.

Example

- In audio, it shows which **frequencies (bass, treble, etc.)** are present in the sound.

This domain helps us understand **what components make up the signal**.

MULTIPLE REPRESENTATIONS OF A SIGNAL



This image uses a music equalizer to explain a fundamental concept in signal processing: **Frequency Domain Representation**.

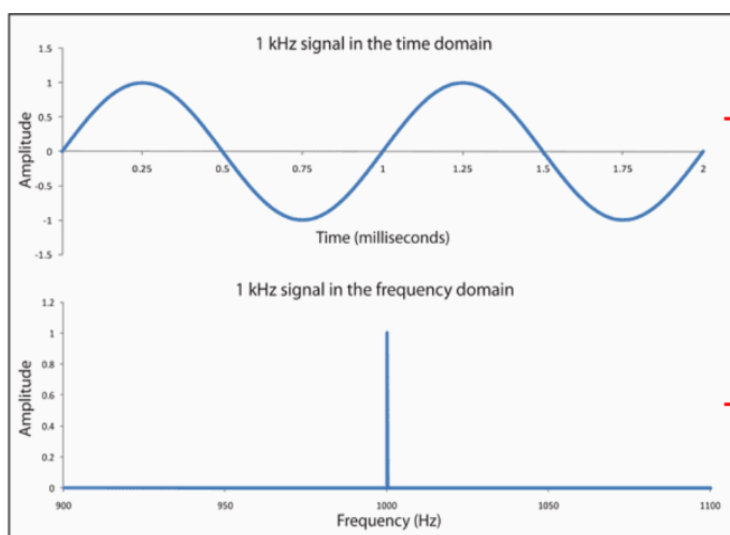
While we usually "see" sound as a single waveform moving through time (the time domain), every complex signal is actually made up of many different individual frequencies layered together.

The Core Concept

The equalizer acts as a bridge between how we hear a song and how it is mathematically structured. Instead of looking at the total volume of the song, it breaks the signal down into its **constituent frequencies**.

- **Low-Frequency Components (Bass):** Shown on the left side of the display (e.g., 31 Hz, 62 Hz). These are the deep, thumping sounds like a kick drum or a bass guitar.
- **Mid-Range Components:** The bars in the center (e.g., 500 Hz to 2K Hz) usually represent vocals and most melodic instruments.
- **High-Frequency Components (Treble):** Shown on the right side (e.g., 8K Hz, 16K Hz). These represent the "brightness" of the sound, like cymbals crashing or the "s" sounds in speech.

Why "Multiple Representations"?



A sine wave oscillates up and down over time, and we can see its amplitude at every moment.

This same sine wave is represented as a single spike at its frequency (e.g., 1000 Hz if it oscillates 1000 times per second).

This image illustrates one of the fundamental concepts in signal processing: the relationship between the **Time Domain** and the **Frequency Domain**.

Essentially, it shows that any signal can be viewed from two different perspectives, much like looking at a musical chord as a wave of sound (time) versus a set of individual notes on a sheet of music (frequency).

Representation	What it shows	Best for...
Time Domain	How the signal changes over time.	Identifying when a sound starts or stops.
Frequency Domain	Which frequencies are present in the signal.	Filtering out noise, boosting bass, or compressing files (like MP3s).

1. The Time Domain (Top Graph)

The top graph shows a **1 kHz sine wave** as it changes over time.

- **What it represents:** It plots **Amplitude vs. Time**. This is what we would see if we looked at the signal on an oscilloscope.
- **Key Characteristic:** We can see the actual oscillation of the wave. Since the frequency is 1 kHz (1,000 cycles per second), one full cycle takes exactly **1 millisecond**.
- **Visual Cue:** The "S" shape shows the signal rising to its peak amplitude, falling to its negative peak, and returning to zero.

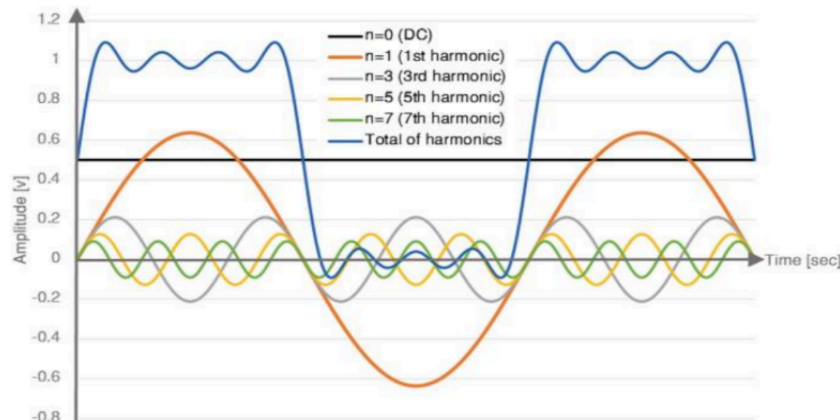
2. The Frequency Domain (Bottom Graph)

The bottom graph shows the exact same signal, but represented through a mathematical "lens" (typically a Fourier Transform).

- **What it represents:** It plots **Amplitude vs. Frequency**.
- **Key Characteristic:** Instead of a waving line, we see a **single sharp spike** (often called a "delta function" or spectral line) located exactly at the **1000 Hz** mark on the horizontal axis.
- **Why it looks like this:** Because this is a "pure" sine wave, it contains only one frequency. In the frequency domain, it doesn't "move" over time; it simply exists as a single value of 1,000 oscillations per second.

The Time Domain

The blue line in the figure below represents a signal that is the summation of multiple sine waves of different frequencies as shown in the image, creating a more complex waveform in the time domain.



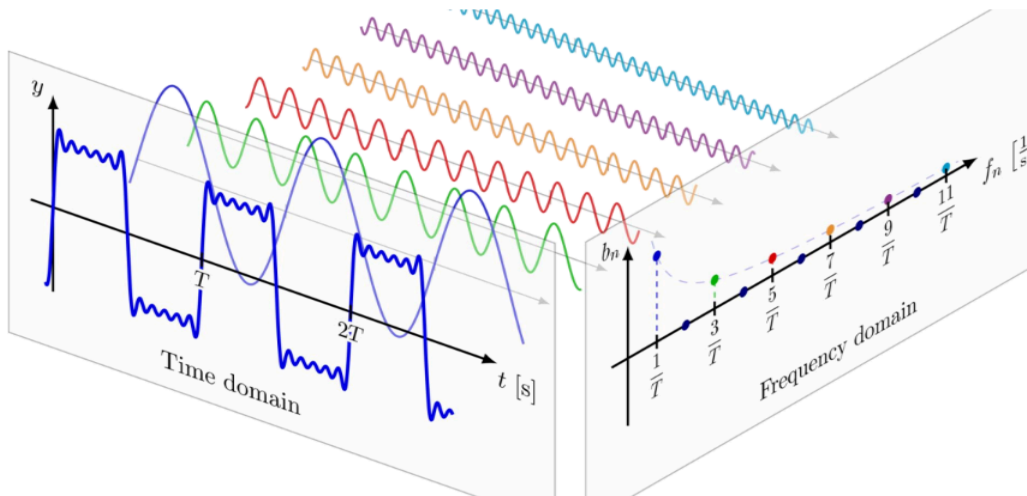
The image shows how a complex wave is "built."

- **Individual Harmonics:** The colorful thin lines ($n = 1, 3, 5, 7$) represent individual sine waves. Notice that as the "n" value increases, the waves wiggle faster (higher frequency) and their height (amplitude) gets smaller.
- **The DC Component ($n=0$):** The solid black horizontal line is the "offset." It shifts the entire wave up from the zero axis.

- **The Composite Signal (Blue Line):** This is the **Summation**. When we add the DC component and all those individual sine waves together at every point in time, we get the thick blue line.
- **The Goal:** Notice how the blue line is starting to look like a "square-ish" wave with some ripples. If we added an infinite number of these harmonics, the blue line would become a perfect square wave.

The Frequency Domain

In the frequency domain, this appears as multiple spikes, one for each frequency component.



34

The image is a 3D visualization that bridges the gap between how we see a signal in time versus how we analyze its "ingredients."

The "Bridge" View

The left side shows the **Time Domain** (Amplitude vs. Time). It's messy because all the waves are overlapping. If we look at the blue signal from the front, we just see one complex shape.

The "Spike" View

The right side shows the **Frequency Domain** (Amplitude vs. Frequency). Imagine rotating the graph 90 degrees to look at it from the side.

- Instead of seeing waves, we see **individual dots or spikes**.
- Each spike represents one of those sine waves from the first image.

- The position on the horizontal axis ($1/T$, $3/T$, $5/T$, etc.) tells us the **frequency** (how fast it wiggles).
- The height of the spike tells us the **strength** (amplitude) of that specific frequency.

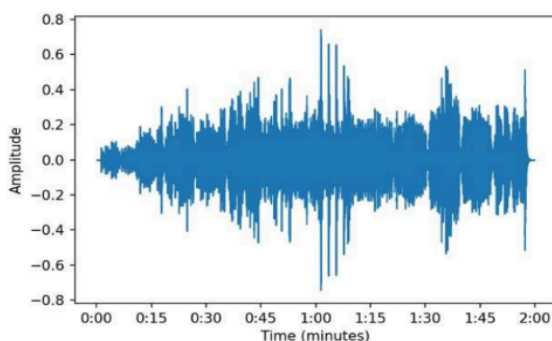
Why does this matter?

This shift in perspective is the "magic" of modern technology:

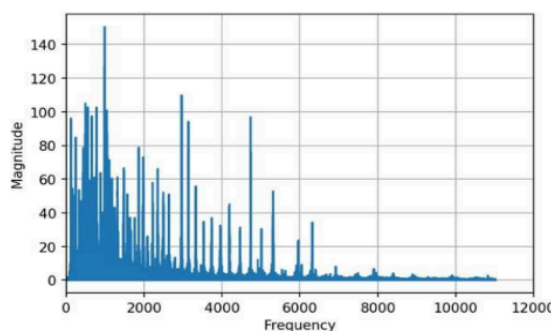
- **Filtering:** If a signal has noise (a high-frequency hiss), it's hard to remove in the Time Domain because it's tangled with the music. In the Frequency Domain, the noise appears as a separate spike that we can simply "cut" out.
- **Bandwidth:** This helps engineers see how much "space" a signal takes up on the radio spectrum or through a fiber optic cable.

Key Takeaway: The Time Domain tells us **when** something happens; the Frequency Domain tells us **what** it is made of.

Most real-world signals, like images or sound, are made up of a combination of many frequencies. So, their frequency domain representation often shows components spanning a significant portion of the frequency spectrum.



**Time Domain Representation
of an Audio Signal**



**Frequency Domain Representation
of the Audio Signal**

This image illustrates one of the fundamental concepts in signal processing: the relationship between the **Time Domain** and the **Frequency Domain**.

Essentially, it shows two different ways to look at the exact same piece of information (in this case, an audio signal).

1. Time Domain Representation (The Left Graph)

This is the view most of us are familiar with- it's what we see when we look at a voice recording on our phone or a song in editing software.

- **X-axis (Time):** Shows the progression of the signal over 2 minutes.
- **Y-axis (Amplitude):** Shows how "loud" or intense the signal is at any given moment.
- **What it tells us:** It's great for seeing **when** things happen. We can see the quiet start, the buildup around the 1-minute mark, and the varying intensity of the sound over time. However, it doesn't tell us anything about the "pitch" or the specific notes being played.

2. Frequency Domain Representation (The Right Graph)

This is created by taking the time-domain signal and running it through a mathematical transformation (usually a **Fast Fourier Transform** or FFT).

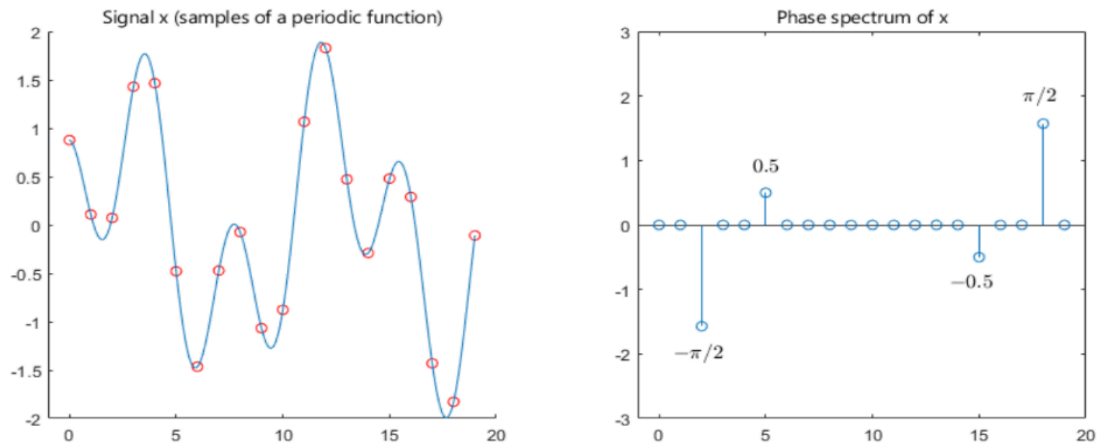
- **X-axis (Frequency):** Shows the range of frequencies (pitches) present in the signal, measured in Hertz (Hz).
- **Y-axis (Magnitude):** Shows the strength or "energy" of each specific frequency.
- **What it tells us:** It's great for seeing **what** the signal is made of. The tall spikes indicate dominant frequencies. For example, if this were a recording of a piano, each spike would represent a specific note being hit. We can see that most of the energy in this signal is concentrated below 6000 Hz.

Why do we need both?

Think of it like a **smoothie**:

- The **Time Domain** is the finished smoothie. We can see how much is in the glass and how the level changes as we drink it.
- The **Frequency Domain** is the recipe. It tells us exactly how much strawberry, banana, and kale went into making that specific taste.

However, the frequency domain representations we have seen so far carry information on the **Magnitude** for each **Frequency**, but not the **Phase**. These are called **Magnitude Spectrums**. Similarly, there is a **Phase Spectrum** that carries information on the **Phase** for each **Frequency**.



The two graphs represent the **exact same signal**, just viewed through different "lenses." To understand how they are the same, we have to look at how a complex signal is built.

1. The Time Domain (Graph 1)

The first graph, **Signal x**, is the **Time Domain** representation. It shows how the signal's amplitude changes as time (or sample index) passes.

- It looks messy and irregular because it isn't just one simple wave; it is the **sum** of multiple sine or cosine waves added together.
- This is what we would see if we looked at an electrical signal on an oscilloscope.

2. The Frequency Domain: Phase (Graph 2)

The second graph, the **Phase Spectrum**, is one-half of the **Frequency Domain** representation (the other half being the **Magnitude Spectrum**, which isn't shown here but is mentioned in the text).

- Instead of time, the horizontal axis represents **Frequency**.
- Each vertical "stalk" (impulse) tells us the **Phase Shift** (ϕ) of a specific frequency component present in the original signal.
- For example, we can see a component at frequency index 2 with a phase of $\pi/2$, and another at index 18 with a phase of $\pi/2$.

FOURIER SERIES & FOURIER TRANSFORM

Sometimes a **complex signal** can be broken into several **simple sine waves** with different frequencies.

But an important question is:

- Can **all signals** be represented like this?
- Even **non-periodic (aperiodic)** signals?

The answer is **Yes**.

This idea comes from the **Fourier Theorem**.

Fourier Theorem

Any signal - continuous or discrete, periodic or aperiodic - can be represented as a sum of sinusoidal waves (sine and cosine) with different frequencies.

These sine and cosine waves act as the **basic building blocks (basis)** of signals in the **frequency domain**.

Why Sinusoids?

Sinusoids (sine and cosine waves) are used because:

- Many **natural physical systems** produce sinusoidal behavior (example: vibrating strings, sound waves, electrical circuits).
- They are **mathematically complete**, meaning combinations of them can represent **any signal**.

Fourier Series

The **Fourier Series** is a method used to represent **periodic signals** using sine and cosine waves.

These sine and cosine waves are called **harmonics**.

The frequencies of these harmonics are **integer multiples of the fundamental frequency**. So, the Fourier Series is basically a **special case of Fourier Theorem** that works only for **periodic signals**.

Fourier Series Formula

$$f(t) = a_0 + \sum_{n=1}^{\infty} [a_n \cos(\frac{2\pi n t}{T}) + b_n \sin(\frac{2\pi n t}{T})]$$

Where,

$$a_k = \frac{1}{T} \int_0^T f(t) \cos(\frac{2\pi k t}{T}) dt \quad \& \quad b_k = \frac{1}{T} \int_0^T f(t) \sin(\frac{2\pi k t}{T}) dt$$

here:

- $a_0 \rightarrow$ Average value (DC component) of the signal
- $a_n \rightarrow$ Amplitude of cosine components
- $b_n \rightarrow$ Amplitude of sine components
- $n \rightarrow$ Harmonic number
- $T \rightarrow$ Period of the signal

This means **any periodic signal can be reconstructed by adding many sine and cosine waves**.

Components of Fourier Series

1. Fundamental Frequency

The basic frequency of the signal.

$f_0 = 1/T$; where T is the period.

2. Harmonics

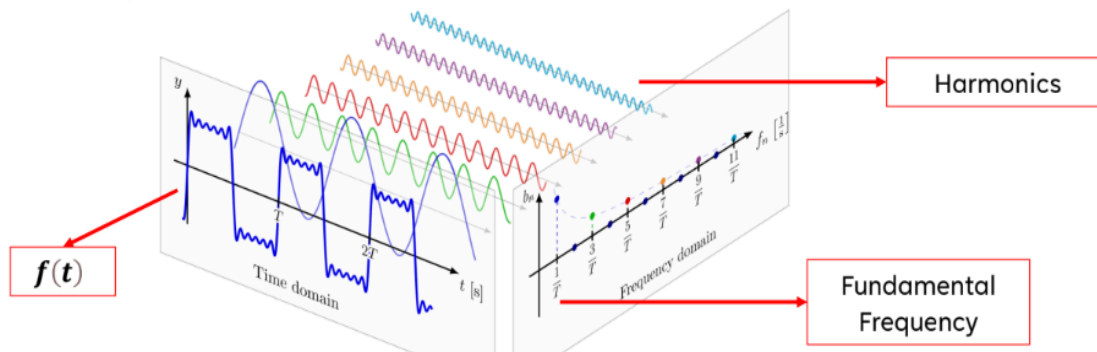
Harmonics are sine waves with frequencies:

$f_0, 2f_0, 3f_0, \dots$

They help capture **more detailed features of the signal**.

3. Coefficients (a_n) and (b_n)

These coefficients indicate **how much of each sine or cosine wave is needed** to rebuild the original signal.



Fourier Transform (FT)

The Fourier Transform converts a signal from the time (or spatial) domain to the frequency domain. Instead of showing how the signal changes with time, it shows what frequencies exist in the signal.

Fourier Transform Formula

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft} dt$$

Where:

- $x(t)$ → Original signal in time domain
- $X(f)$ → Signal in frequency domain
- t → Time (or spatial coordinate)
- f → Frequency

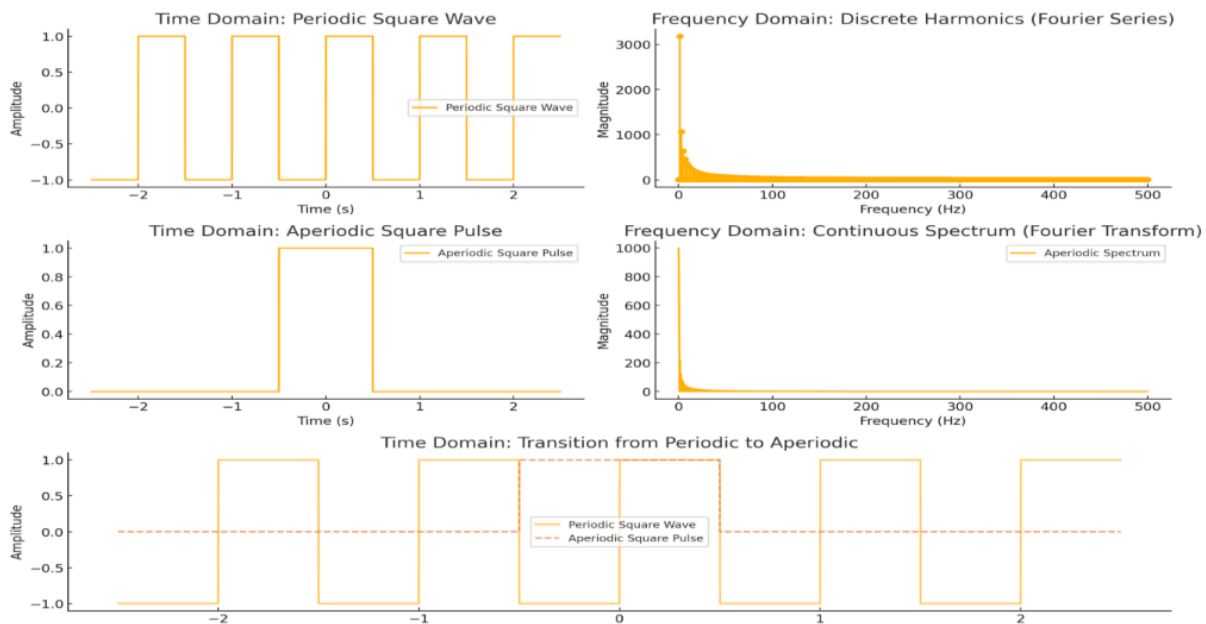
The Fourier Transform represents a signal as **a combination of infinitely many sinusoidal waves**.

Meaning of Frequency Domain

The **frequency domain** tells us:

- What **frequencies are present** in the signal
- How **strong** each frequency is

The magnitude of $X(f)$ shows the **energy of the signal at each frequency**. So Fourier Transform also shows the **energy distribution of the signal**.



1. The Fourier Series (Top Row)

The top row shows a **Periodic Square Wave**. Because the signal repeats forever, its energy is concentrated at specific, "allowed" frequencies- the fundamental frequency and its harmonics.

- **Time Domain:** A repeating pattern with a fixed period T .
- **Frequency Domain:** We see **Discrete Harmonics** (the vertical orange stems). This is like a musical chord; we can clearly identify the individual "notes" that make up the signal.

2. The Fourier Transform (Middle Row)

The middle row shows an **Aperiodic Square Pulse**. This is a single "blip" that never repeats.

- **Time Domain:** A single pulse at $t=0$, flat everywhere else.
- **Frequency Domain:** Notice the shift from dots to a smooth line. This is a **Continuous Spectrum**. Because the signal doesn't repeat, we need a "little bit" of every possible frequency to describe it.

3. The Conceptual Bridge (The Transition)

The bottom plot explains the mathematical "magic" used to get from one to the other. To turn a Fourier Series into a Fourier Transform, we imagine the period T of the square wave getting longer and longer.

- As $T \rightarrow \infty$, the distance between the next pulse becomes infinite.
- In the frequency domain, as the pulses get further apart in time, the harmonics get closer and closer together.
- Eventually, the "gaps" between the harmonics vanish, and the discrete stems merge into the smooth, continuous curve we see in the middle-right plot.

Inverse Fourier Transform

The **Inverse Fourier Transform** converts the signal **back to the time domain**.

Inverse Fourier Transform Formula

$$x(t) = \int_{-\infty}^{\infty} X(f) e^{j2\pi ft} df$$

This reconstructs the **original signal** from its frequency components.

Applications of Fourier Transform

Fourier methods are widely used in many fields:

Audio Processing

- Speech recognition
- Noise removal

Image Processing

- Texture analysis
- Image compression

Machine Learning

- Feature extraction
- Noise reduction
- Data representation

Common optimized versions include:

- FFT (Fast Fourier Transform)
- STFT (Short-Time Fourier Transform)
- DCT (Discrete Cosine Transform)

These methods make Fourier calculations **much faster and more efficient**.

Why is $e^{j2\pi ft}$ Used?

The term $e^{j2\pi ft}$ comes from **Euler's Formula**:

$$e^{j\theta} = \cos \theta + j \sin \theta$$

This means a **complex exponential represents both sine and cosine waves together**.

So using $e^{j2\pi ft}$:

- simplifies calculations
- represents **sinusoidal signals compactly**
- makes Fourier analysis mathematically easier.

Discrete Fourier Transform (DFT)

The Discrete Fourier Transform (DFT) is a mathematical method used to convert discrete data from the time (or spatial) domain into the frequency domain.

It is very important in Digital Signal Processing and Image Processing because computers work with discrete values (sampled data or pixel values).

In simple terms, DFT helps us find the frequency components present in a digital signal.

Discrete Signal

In DFT, we work with a **finite sequence of samples**:

$f[n]$, $n = 0, 1, 2, \dots, N-1$

Where:

- $f[n]$ → value of the signal at sample n
- N → total number of samples

DFT Formula

$$F[k] = \sum_{n=0}^{N-1} f[n] e^{-j2\pi kn/N}, \quad k = 0, 1, 2, \dots, N-1$$

Where

- $f[n]$ → original signal in time or spatial domain
- $F[k]$ → frequency representation of the signal
- n → sample index (time or space)
- k → frequency index
- N → total number of samples

So DFT converts **N time-domain samples** into **N frequency components**.

Meaning of the Result

After applying DFT:

- Each **F[k]** represents a **specific frequency component**.
- The **magnitude** of F[k] shows **how strong that frequency is** in the signal.

So DFT tells us:

- Which frequencies exist
- How much of each frequency is present

Inverse Discrete Fourier Transform (IDFT)

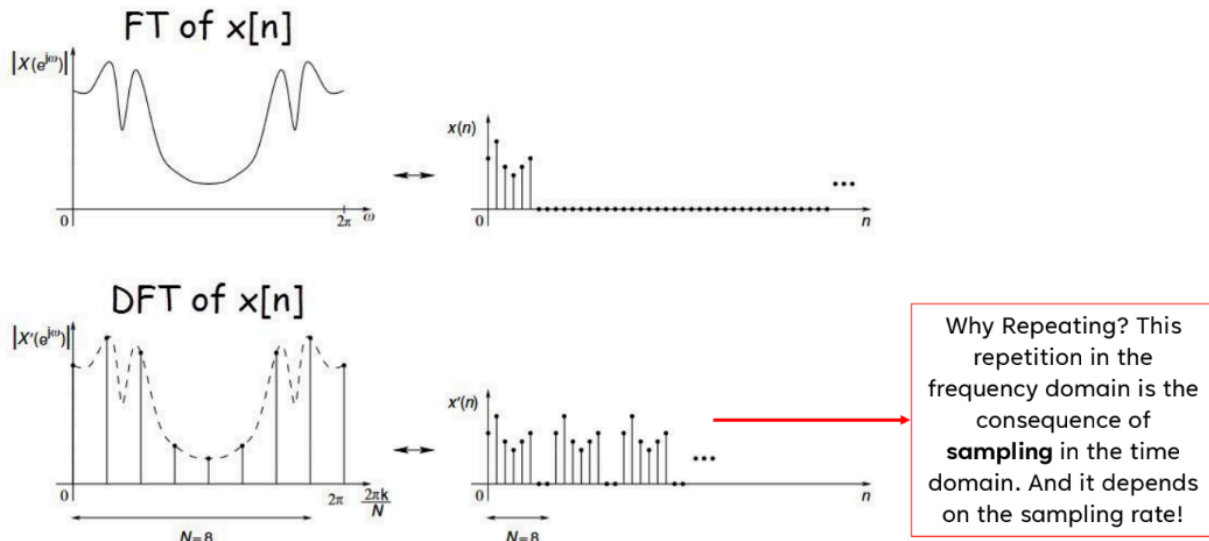
The **Inverse DFT** converts the signal **back from the frequency domain to the time domain**. It reconstructs the **original signal**.

IDFT Formula

$$f[n] = \frac{1}{N} \sum_{k=0}^{N-1} F[k] e^{\frac{j2\pi kn}{N}}$$

Where

- **F[k]** → frequency-domain representation
- **f[n]** → reconstructed original signal
- **N** → number of samples



This image highlights the fundamental relationship between the **Discrete Fourier Transform (DFT)** and its continuous-time counterpart. The repetition we see in the bottom graph is the "cost" of discretization.

1. The Core Concept: Duality

In signal processing, there is a "bridge" between the time and frequency domains. A key rule of this bridge is:

- Sampling in one domain causes periodicity in the other.

2. Breaking Down the Image

Top Row: Discrete-Time Fourier Transform (DTFT)

- **Time Domain ($x[n]$):** We have a single, finite sequence of pulses. It starts, it ends, and it is zero everywhere else (aperiodic).
- **Frequency Domain ($|X(e^{j\omega})|$):** Because the time signal is aperiodic, its frequency spectrum is **continuous**. However, because the time signal is discrete (sampled), the spectrum is already periodic with a period of 2π .

Bottom Row: Discrete Fourier Transform (DFT)

- **Frequency Domain ($|X'(e^{j\omega})|$):** To compute this on a computer, we cannot have a "continuous" curve. We must **sample** the frequency spectrum at N points (here, $N=8$).

- **Time Domain ($x'(n)$):** This is where the magic (and the confusion) happens. When we sample the frequency spectrum, the math of the Inverse DFT forces the time-domain signal to become **periodic**.

3. Why is it repeating?

The red box in our image correctly notes that this is a consequence of sampling. Mathematically, the DFT treats the signal as if it lives on a circle.

When we take an N-point DFT, we are essentially saying: *"I am assuming this signal repeats every N sample forever."* If our original signal $x[n]$ was longer than N, these repetitions would overlap, causing **aliasing**. Since our signal is finite and fits within $N=8$, the repetitions (shown as $x'(n)$) stay distinct, but they must exist for the discrete math to remain consistent.

Frequency in Spatial Domain

To understand frequency in the **spatial domain**, first consider a **simple sinusoidal signal in the time domain**.

Sinusoidal Signal in Time Domain

A sinusoidal signal can be written as: $A\sin(2\pi f_0 t + \theta)$

Where:

- $A \rightarrow$ Amplitude (strength of the signal)
- $f_0 \rightarrow$ Frequency (how fast the signal oscillates)
- $t \rightarrow$ Time (independent variable)
- $\theta \rightarrow$ Phase (shift of the signal)

In the **time domain**, the signal is described by **amplitude changing over time**.

Frequency Domain Representation

The same signal can also be represented in the **frequency domain**.

In the frequency domain, the sinusoid is represented by **two impulses at frequencies $+f_0$ and $-f_0$** .

$$X(f) = 2A\delta(f-f_0) + 2A\delta(f+f_0)$$

This representation shows:

- Which frequencies exist
- How strong they are (amplitude)

So the **magnitude spectrum** contains information about:

- Frequency
- Amplitude

The **phase spectrum** contains information about:

- Phase shift of the signal.

Sinusoid in Spatial Domain (1D)

In the **spatial domain**, the structure of the sinusoidal function **does not change**. The only difference is the **independent variable**. Instead of **time (t)**, we use **space or position (x)**.

$$A\sin(2\pi f_0 x + \theta)$$

Where:

- $A \rightarrow$ Amplitude
- $f_0 \rightarrow$ Spatial frequency
- $x \rightarrow$ Spatial position (distance)
- $\theta \rightarrow$ Phase

Key Idea

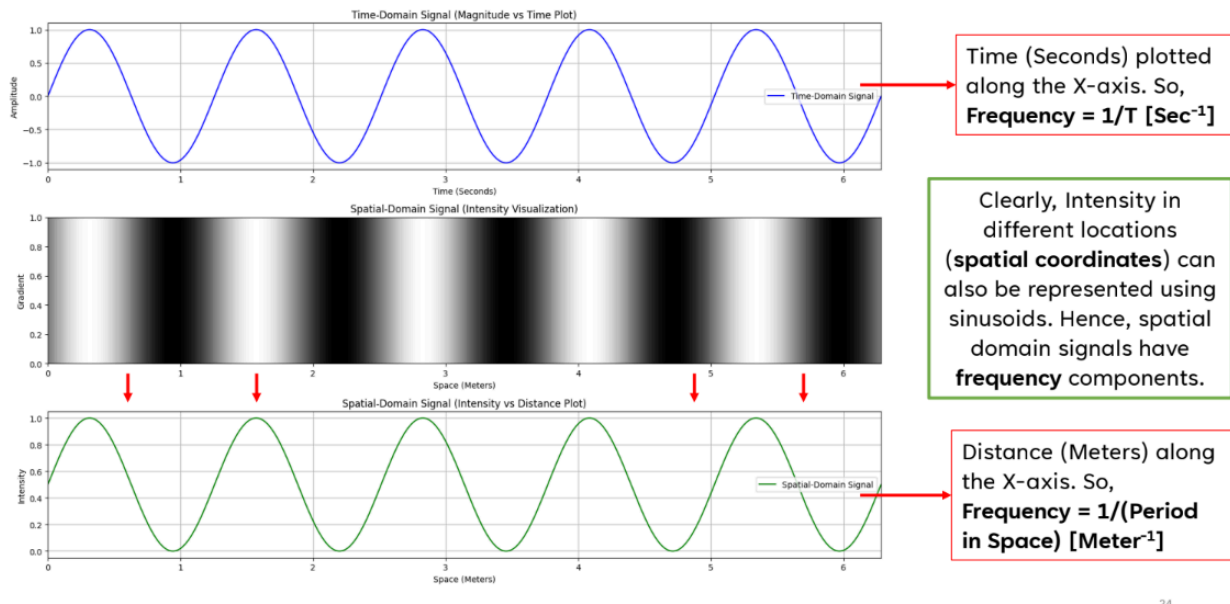
- In **time domain signals**, the independent variable is **time (t)**.
- In **spatial domain signals**, the independent variable is **space or position (x)**.

The concept of **frequency** is **the same**, but in spatial signals it represents **how quickly the intensity changes across space**.

Example:

- **Low spatial frequency** → smooth image with gradual changes
- **High spatial frequency** → sharp edges and fine details in images.

FREQUENCY IN SPATIAL DOMAIN



1. Time-Domain Signal (Top)

This is the standard way we think about frequency.

- **The Plot:** Shows a blue sine wave where the horizontal axis represents **Time (Seconds)**.
- **The Math:** If it takes T seconds for one full cycle to complete, the frequency is $f = 1/T$, measured in **Hertz (Sec^{-1})**.
- **Intuition:** It tells us how fast the signal changes over *time*.

2. Spatial-Domain Visualization (Middle)

This bridges the gap between a "wave" and an "image."

- **The Visual:** Instead of a line, we see a gradient of black and white bars.
- **The Connection:** Notice how the brightest white areas correspond to the peaks of the wave below, and the black areas correspond to the troughs.
- **Intuition:** In digital image processing, a "signal" isn't a sound; it's the **intensity (brightness)** of pixels across a surface.

3. Spatial-Domain Signal (Bottom)

This treats a physical image like a mathematical wave.

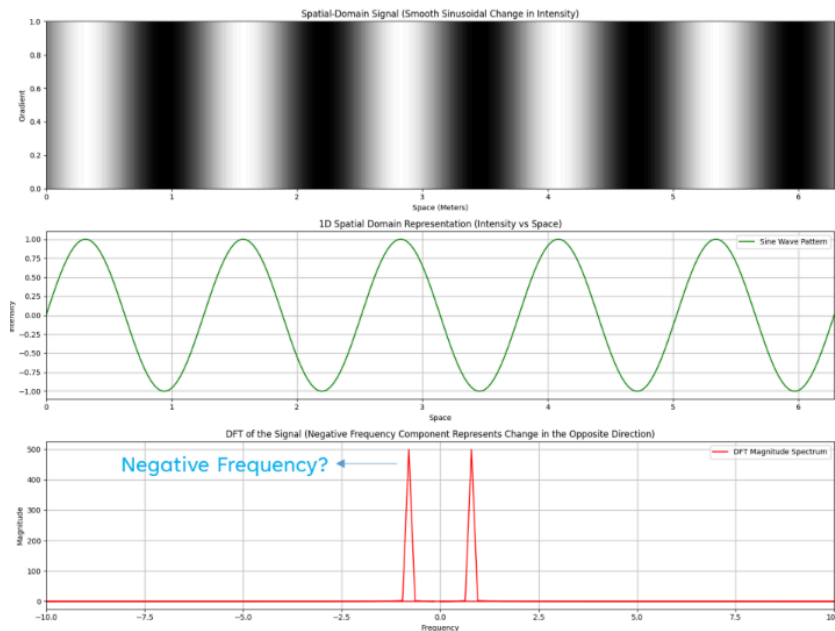
- **The Plot:** Shows a green sine wave where the horizontal axis represents **Space (Meters)**.
- **The Math:** Instead of a time period T , we have a **spatial period** (the distance between two peaks). The spatial frequency is calculated as $1/\text{Distance}$, measured in **Meter⁻¹** (cycles per meter).
- **Intuition:** High spatial frequency means the image details change very rapidly over a short distance (like fine stripes or sharp edges). Low spatial frequency means the intensity changes slowly (like a smooth sky or a blurry shadow).

Why does this matter?

Understanding "frequency in space" is the foundation for **Image Filtering**.

- **Low-pass filters** blur an image because they remove high spatial frequencies (sharp changes).
- **High-pass filters** sharpen an image or perform edge detection because they keep only the rapid changes in intensity.

FREQUENCY IN SPATIAL DOMAIN: 1D



The image here shows the change in intensity along one direction following a sinusoidal pattern. Expectedly, in the DFT, there is a spike at around **0.8**, which is the frequency of the original sinusoid.

The spike at around **-0.8** is due to the **complex conjugate symmetry** of the transform for real-valued signals (remember the basis?). These negative frequencies represent the same information as the positive ones but in the opposite direction. For real signals, only the **positive frequencies** are typically considered for analysis.

1. The Spatial Domain Signal (Top)

The top grayscale bar represents a 1D image. We can see the intensity (brightness) changing smoothly from black to white. In image processing, we view this as a **signal**.

- **The Pattern:** Because the bars repeat at regular intervals, it has a specific "spatial frequency."
- **The Visualization:** The smooth transition indicates it is a pure sinusoid rather than sharp, jagged edges (which would contain many higher frequencies).

2. 1D Spatial Representation (Middle)

This green plot is the mathematical representation of the grayscale bar above it.

- **Y-axis (Intensity):** High peaks correspond to the white bars; low troughs correspond to the black bars.
- **X-axis (Space):** This shows how the intensity fluctuates as we move across the image from left to right.

- **The Frequency:** If we count the cycles over the space provided, we get a frequency of approximately **0.8 cycles per unit of space**.

3. The DFT Magnitude Spectrum (Bottom)

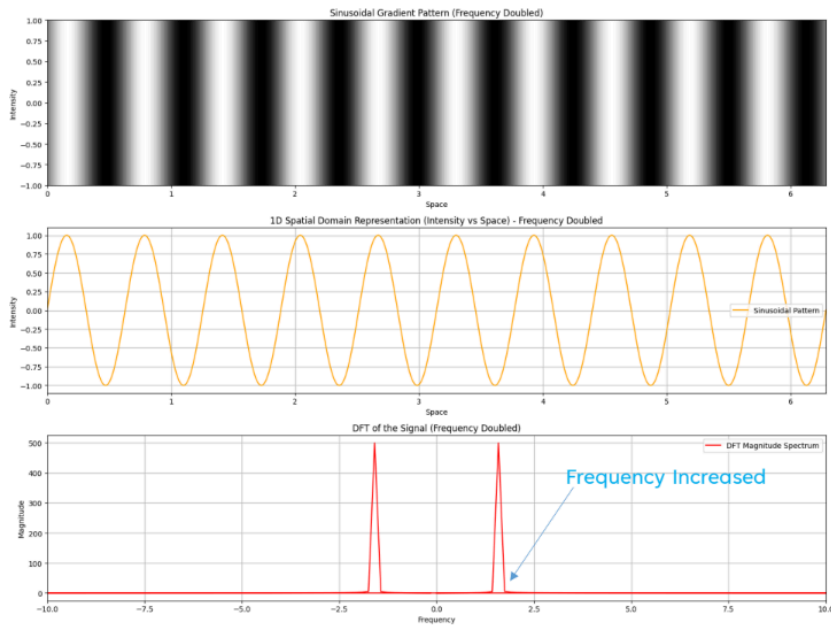
This is the result of applying the **Discrete Fourier Transform (DFT)** to the signal. It moves the data from the "Spatial Domain" (where things are located) to the "Frequency Domain" (how fast things change).

- **The Spike at 0.8:** This is the "Main Event." It tells us that the primary pattern in the image is a wave repeating at a frequency of 0.8.
- **The Spike at -0.8 (Negative Frequency):** This often confuses us. Mathematically, the Fourier Transform uses complex exponentials (Euler's formula). For any real-valued signal (like a standard photo), the transform is **symmetrical**.
 - The negative frequency is a "mirror image" required by the math to ensure that when we transform back to the spatial domain, the imaginary parts cancel out, leaving us with a real image.
 - As the text notes, in practical image analysis, we usually focus on the positive side because the negative side provides redundant information.

Why does this matter for images?

Understanding this 1D example is the "building block" for 2D image processing.

- **Low Frequencies:** Represent smooth areas or slow transitions in an image (like a clear blue sky).
- **High Frequencies:** Represent sharp details, edges, or noise (like the texture of a brick wall).



The image here shows the aftermath of doubling the rate of change in intensity along the spatial axis. Expectedly, in the DFT, the spike is now at around **1.6**, which is double the frequency of the original sinusoid.

Notice that while the signal is compressing in the time domain, it is stretching in the frequency domain (and vice versa for a lower-frequency signal).

1. Spatial Domain (Top & Middle)

- **Visual Pattern:** Shows a grayscale "Sinusoidal Gradient Pattern" where intensity fluctuates between black (-1.0) and white (1.0).
- **Waveform:** A 1D plot of intensity vs. space. The signal is a pure sine wave that has been **frequency doubled**.
- **Compression:** Doubling the frequency compresses the signal in the spatial domain (more peaks and valleys within the same space).

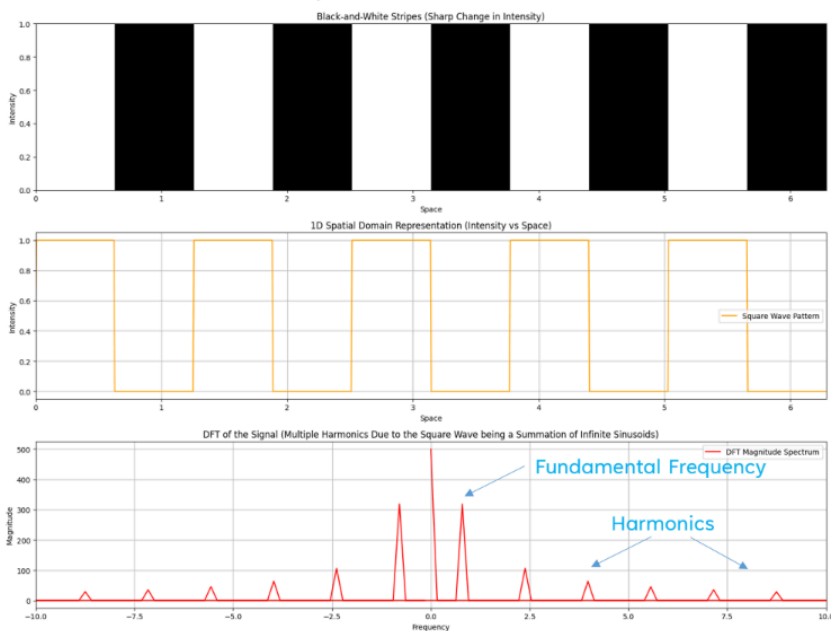
2. Frequency Domain / DFT (Bottom)

- **The Spikes:** The Discrete Fourier Transform (DFT) shows two sharp peaks (spikes) at approximately ± 1.6 on the frequency axis.
- **Magnitude:** These spikes represent the dominant frequency component of the spatial signal.
- **Key Concept:** The spikes have shifted further away from the center (0) compared to a lower-frequency signal.

3. Inverse Relationship

- **Scaling Property:** The text highlights a fundamental rule: **compression in the spatial domain equals stretching in the frequency domain.**

- **Inverse Effect:** If the signal frequency were lowered, the peaks in the DFT would move closer to zero (the center).



In this case, The change in intensity along the spatial axis does not follow sinusoidal functions. Rather, the intensity is changed abruptly from high to low periodically. The 1D spatial domain representation of this is a **square wave**.

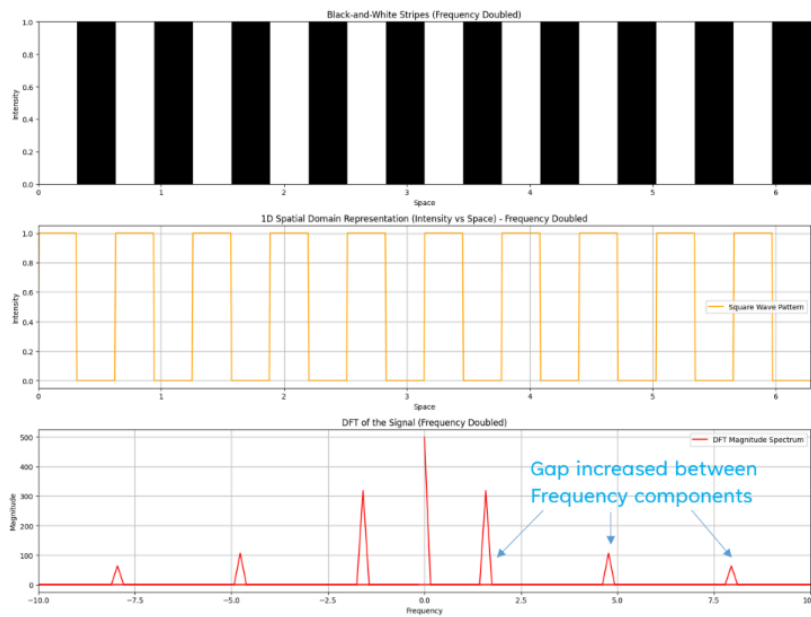
Here, apart from the fundamental frequency f_0 at around 0.8, we can see the harmonics too at $3f_0$, $5f_0$, etc. This is because a **perfect square wave can be represented as a summation of infinite sinusoids**. That's why we can see the frequency components of all those sinusoids here.

27

This slide illustrates how sharp changes in an image (like stripes) are represented in the frequency domain using a **1D Square Wave**.

Key Concepts

- **Spatial Domain (Top/Middle):** A pattern of black-and-white stripes creates abrupt intensity changes. This is represented mathematically as a **square wave** rather than a smooth sinusoid.
- **Frequency Domain (Bottom):** The Discrete Fourier Transform (DFT) of this square wave shows multiple peaks:
 - **Fundamental Frequency (f_0):** The primary rate at which the pattern repeats (located at approximately 0.8 in this example).
 - **Harmonics:** Additional peaks appearing at odd multiples of the fundamental ($3f_0$, $5f_0$, etc.).
- **The "Why":** A perfect square wave is the summation of an **infinite series of sinusoids**. Therefore, its frequency spectrum must contain all those higher-frequency components to account for the "sharpness" of the edges.



Once again, we can see the aftermath of doubling the rate of change in intensity along the spatial axis. Expectedly, in the DFT, the spike for the fundamental frequency is now at around **1.6**, which is double the frequency of the previous square wave.

As we can see, higher spatial frequencies correspond to closer spacing in the pattern itself, yet their Fourier Transform produces spikes that are further apart. It is because if the spatial pattern has a high frequency (denser lines, closer together), the wavelength λ of the pattern is smaller.

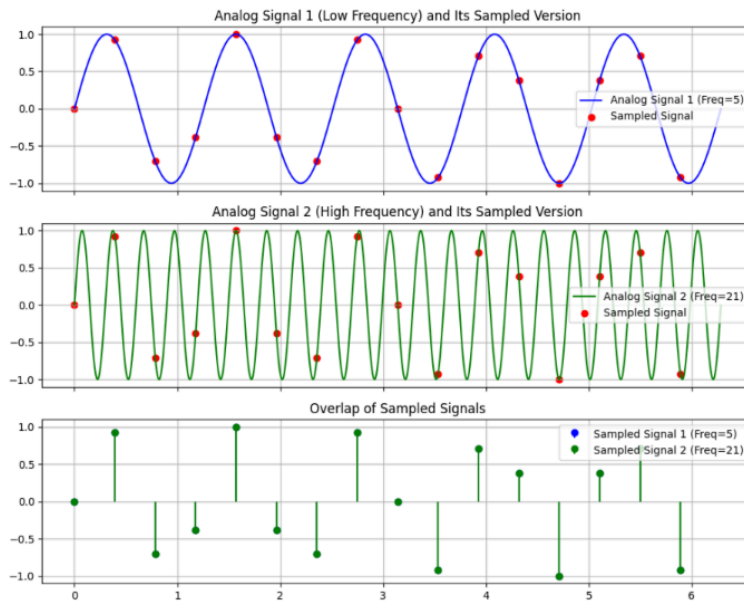
Top Plot (Visual): Displays the physical stripes; the pattern repeats roughly every 0.6 units of space.

Middle Plot (Waveform): A square wave representation showing intensity switching between 0.0 and 1.0.

Bottom Plot (DFT): The Discrete Fourier Transform showing spikes at specific frequencies.

- **Fundamental Frequency:** Located at approximately **1.6**.
- **Harmonics:** Additional spikes appear at odd multiples (approx. 4.8, 8.0, etc.), which is characteristic of a square wave.
- **Gap Increase:** Because the frequency was doubled, the "gap" between these spikes on the x-axis has increased.

FREQUENCY IN SPATIAL DOMAIN: SAMPLING



DFT can't capture every frequency component in the spectrum. The reason behind that is the **Sampling Rate**. Observe the sampled signals here (spanning 1 second/meter).

Despite the difference in frequency and values, the sampled outputs for both signals 1 and 2 are identical. Hence, it's impossible to recover both signals. That's why the sampling rate must be at least **twice** the highest frequency present in the signal, as per the **Nyquist Theorem**. Frequencies above half the sampling rate get folded back into the range below it, which is what happens to signal 2 here (it gets lost), and signal 1 is reconstructed.

This picture illustrates the critical relationship between **sampling rates** and **signal reconstruction** in the spatial domain. Here are the important takeaways:

Core Concepts

- **Aliasing:** When a sampling rate is too low, high-frequency signals (Signal 2) "fold back" and appear identical to low-frequency signals (Signal 1). This makes it impossible to distinguish between them after sampling.
- **Nyquist Theorem:** To successfully recover a signal, the sampling rate must be **at least twice** the highest frequency present in that signal ($f_s > 2f_{max}$).

Comparison Table

Feature	Analog Signal 1	Analog Signal 2
Frequency	Low (Freq = 5)	High (Freq = 21)

Status	Correctly sampled	Aliased (lost)
Sampled Output	Identical to Signal 2	Identical to Signal 1

Key Observations

- **The Problem:** The Discrete Fourier Transform (DFT) is limited by the sampling rate; it cannot capture frequencies that exceed half of that rate.
- **The Visualization:** The bottom graph shows the "Overlap of Sampled Signals." Because the red dots land on the same values for both waves, the high-frequency data is effectively lost and replaced by the lower-frequency "alias."

Key Insights

1. Negative Frequencies in DFT

When we apply the **Discrete Fourier Transform (DFT)** to real signals, we often see **negative frequency spikes**.

- This happens because of **complex conjugate symmetry** in the DFT.
- Negative frequencies contain **the same information as positive frequencies**, but in the **opposite direction**.
- Because they repeat the same information, they are **usually ignored in analysis**.

2. Spatial Frequency and Pattern Spacing

There is a direct relationship between **spatial frequency** and **spacing between patterns**.

- **Closer lines or patterns** → intensity changes quickly → **high spatial frequency**
- **Wider spacing** → intensity changes slowly → **low spatial frequency**

In the frequency domain:

- **High spatial frequencies produce larger frequency values.**

Example:

- **Sharp edges** in images correspond to **high frequencies**.

3. Harmonics in Signals

A **square wave** can be represented as a sum of **many sinusoidal waves**.

These sinusoids have frequencies that are **odd multiples of the fundamental frequency**.

Example:

$f_0, ; 3f_0, ; 5f_0, ; 7f_0 ..$

These are called **harmonics**.

Most **real-world signals** contain many sinusoids, so they have **multiple frequency components**.

4. Intensity Changes and Frequency

- Frequent changes in intensity → higher spatial frequencies
- Slow intensity changes → lower spatial frequencies

In images:

- **Dense patterns** → high spatial frequency
- **Smooth regions** → low spatial frequency

In the frequency domain:

- High spatial frequencies appear as **spikes farther from the center**.

5. Nyquist Frequency and Aliasing

The **DFT can only detect frequencies up to half of the sampling rate**.

This limit is called the **Nyquist Frequency**.

$$f_{\text{Nyquist}} = f_s / 2$$

Where:

- f_s = sampling frequency.

If a signal contains frequencies **higher than this limit**, they will **fold back into the lower frequency range**.

This effect is called **aliasing**.

Another important point:

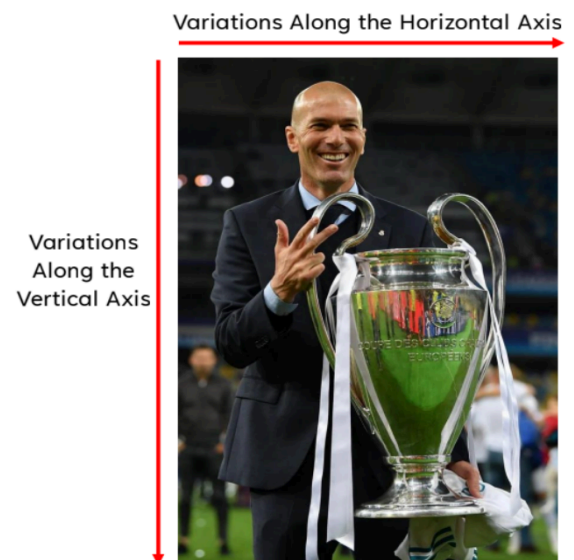
- The **DFT spectrum is periodic** and repeats every 2π .

FREQUENCY IN 2D IMAGES

1. From 1D to 2D Signals

- **1D signal** (like a sine wave):
 - Varies in **one direction** (time or space)
 - Example: audio signal, waveform
- **2D signal** (image):
 - Varies in **two directions**:
 - **x-axis** (horizontal → columns)
 - **y-axis** (vertical → rows)

So, an image is basically a function: $f(x, y)$



2. Meaning of Frequency

- In **1D**:
 - Frequency = how fast the signal changes (oscillates)
- In **2D images**:
 - Frequency = how quickly **pixel intensity changes** across the image

3. Types of Frequency in 2D

Horizontal Frequency

- Measures changes **along the x-direction**
- High horizontal frequency → rapid changes left to right
- Example:
 - **Vertical stripes** (because intensity changes as we move horizontally)

Vertical Frequency

- Measures changes **along the y-direction**
- High vertical frequency → rapid changes top to bottom
- Example:
 - **Horizontal stripes**

4. Combined Effect

- Real images are not just horizontal or vertical patterns
- They contain **combinations of both frequencies**

Together, horizontal + vertical components can represent:

- Diagonal edges
- Textures
- Complex patterns

5. Key Intuition

- **Low frequency**:

- Smooth regions (sky, plain background)
- Slow intensity change
- **High frequency:**
 - Edges, sharp details, noise
 - Rapid intensity change

6. Visual Understanding

Pattern Type	Frequency Type
Smooth surface	Low frequency
Vertical stripes	High horizontal freq
Horizontal stripes	High vertical freq
Edges/details	High frequency

7. Final Insight

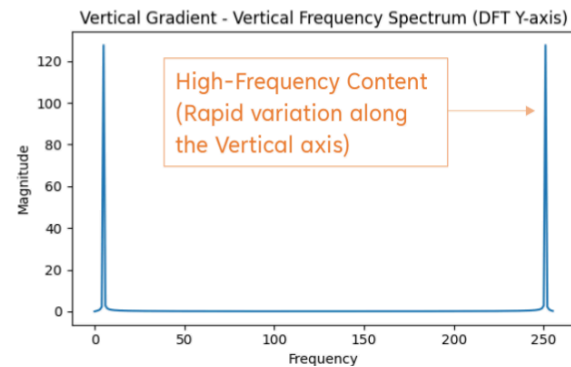
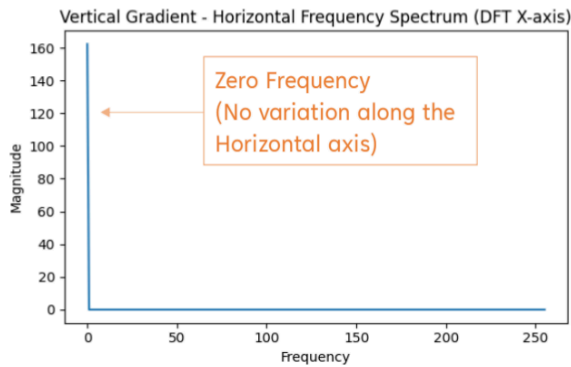
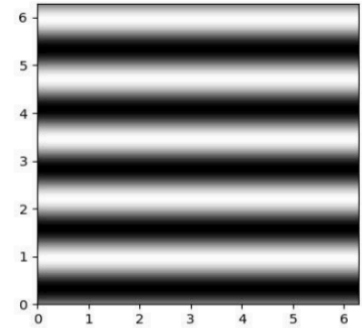
Any 2D image can be represented using combinations of:

- Horizontal frequencies
- Vertical frequencies

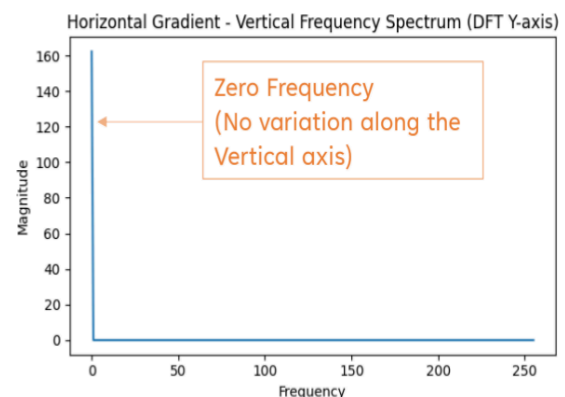
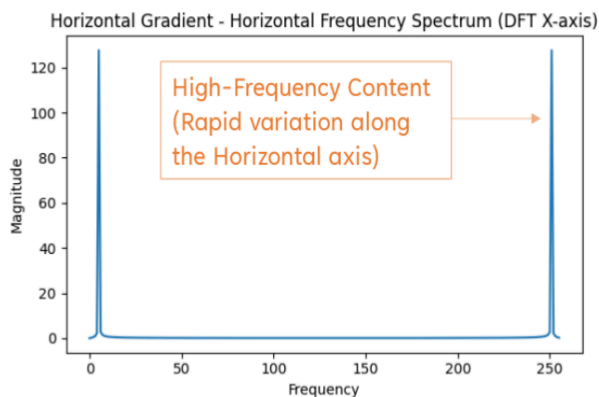
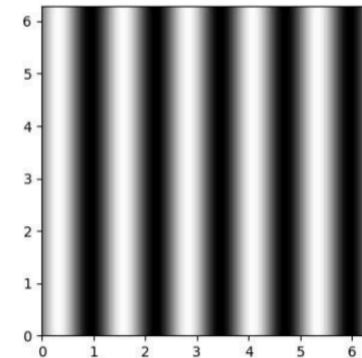
This is the foundation of transforms like:

- **2D Fourier Transform**
- Image filtering (blur, sharpening, edge detection)

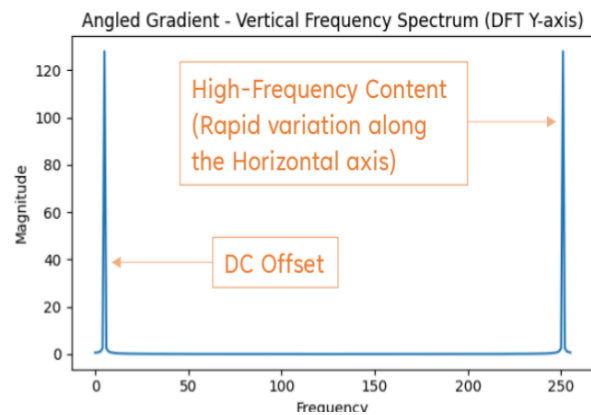
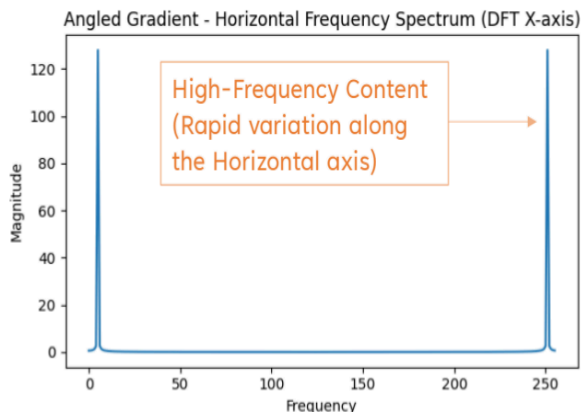
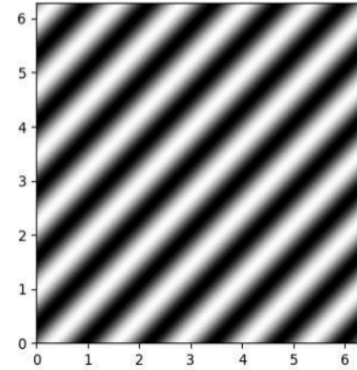
Let us take a look at the image here and the DFTs along the horizontal (X) and Vertical (Y) axis below. As we can see, there is no frequency component (apart from the DC component) in the horizontal frequency spectrum (since there's no change in intensity along the X-axis). But there's a high-frequency component in the vertical frequency spectrum as intensity changes periodically along the Y-axis.



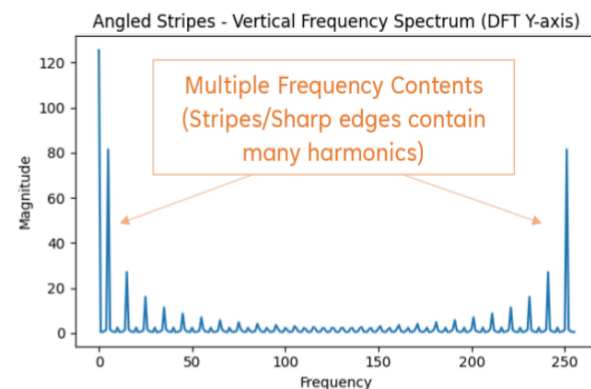
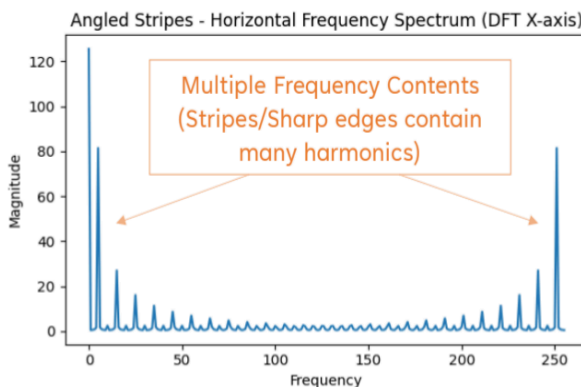
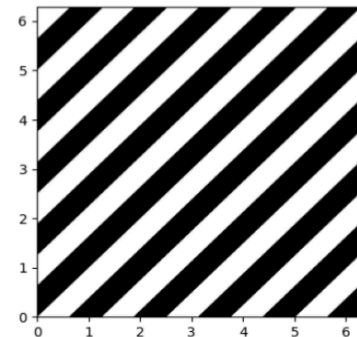
Now take a look at the image here and the DFTs along the horizontal (X) and Vertical (Y) axis below. As we can see, there is no frequency component (apart from the DC component) in the vertical frequency spectrum (since there's no change in intensity along the Y-axis). But there's a high-frequency component in the horizontal frequency spectrum as intensity changes periodically along the X-axis.



Now take a look at the image here and the DFTs along the horizontal (X) and Vertical (Y) axis below. As we can see, there's a high-frequency component in both cases, the horizontal frequency spectrum, and the vertical frequency spectrum, as intensity changes periodically along both the X-axis and the Y-axis.



Finally, take a look at the image here and the DFTs along the horizontal (X) and Vertical (Y) axis below. As we can see, there are many frequency components in both cases, the horizontal frequency spectrum, and the vertical frequency spectrum, as intensity changes periodically along both the X-axis and the Y-axis. The various frequencies are due to the sharp edges in this image that contain many frequencies.



HOW MANY DFTS?

So, we can plot the frequency contents along both axes of an image. However, there are many horizontal and vertical lines within an image. It is unrealistic to plot so many 1D DFTs to analyze the frequency contents of an image. Then how can we represent an image in the frequency domain properly? The answer is in 2D Discrete Fourier Transform (DFT).

2D DISCRETE FOURIER TRANSFORM (DFT)

The 2D Discrete Fourier Transform (DFT) is an extension of the 1D DFT to two dimensions, typically applied to 2D signals like images. It transforms a spatial-domain representation (pixel intensities) into the frequency domain, capturing the frequency components in both horizontal and vertical directions. For a 2D signal $f(x, y)$ of size $M \times N$, the 2D DFT is given by:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

Where,

- $f(x, y)$ is the Intensity at spatial position x, y .
- $F(u, v)$ is the Frequency component at (u, v) .
- u, v are the horizontal and vertical frequency indices, respectively.
- M, N represent the dimensions of the image (Number of pixels along the x & y axis).

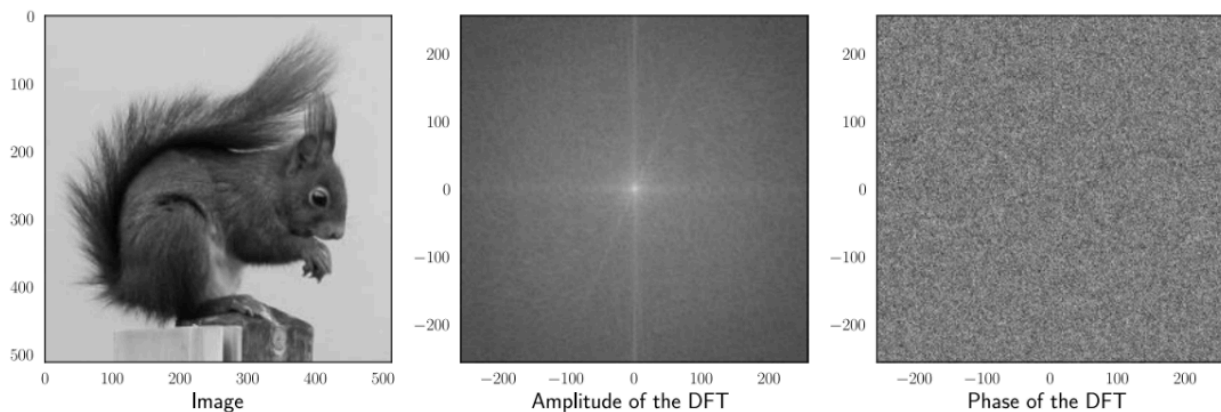
Clearly, the value of 2D DFT $F(u, v)$ at each frequency index u, v is a complex number. So,

$$F(u, v) = a + bj = r \angle \theta$$

Where,

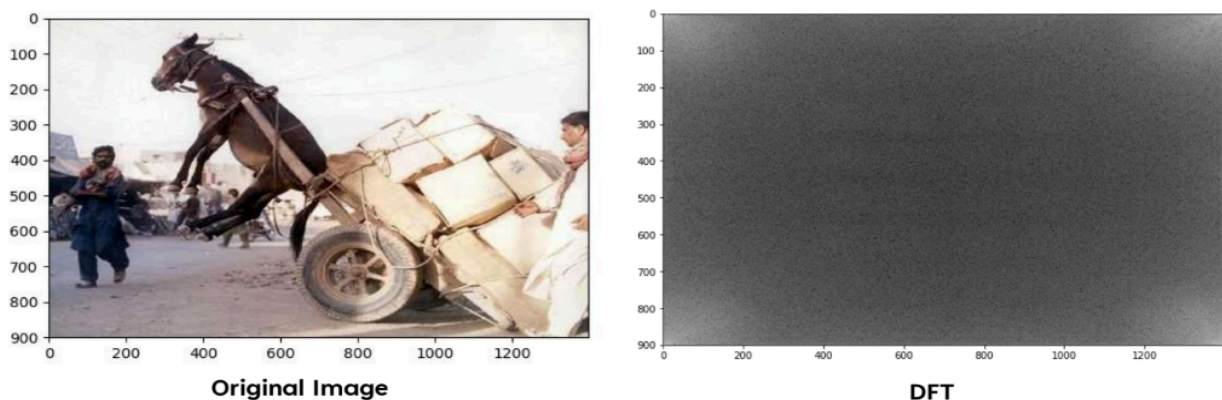
- $r = |F(u, v)|$ is the magnitude/amplitude.
- θ represents the phase.

These two quantities for all frequency indices correspond to Magnitude Spectrum and Phase Spectrum respectively. These 2D image-like spectrums show higher intensities for larger values.

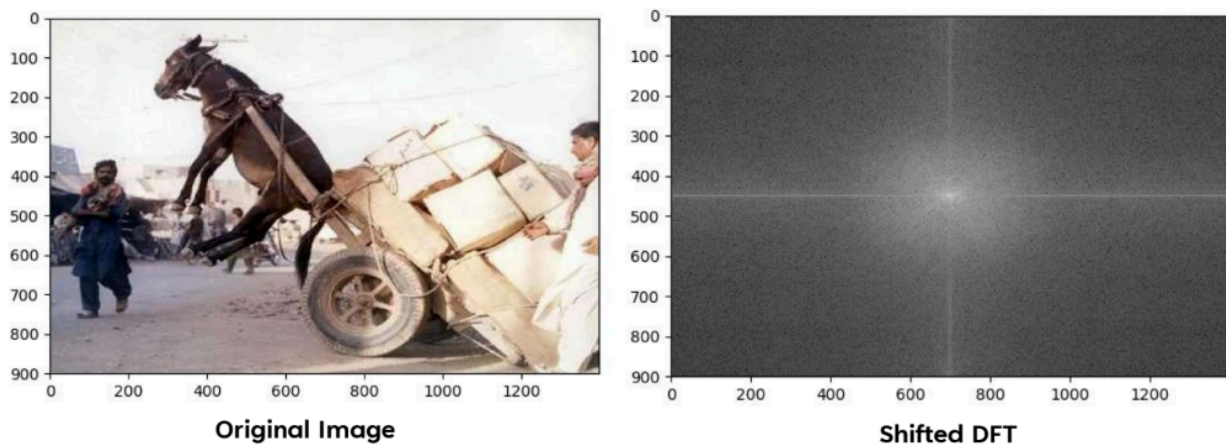


2D DFT: SHIFTING THE ORIGIN

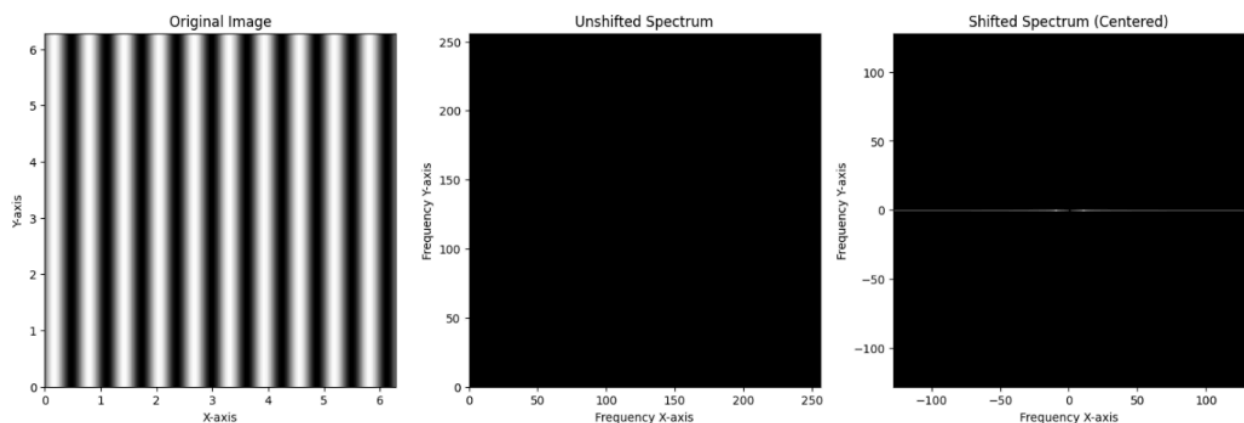
In 2D DFT, by default, the low-frequency components (representing overall trends or smooth variations) are at the corners of the spectrum, while the high-frequency components (representing fine details or sharp edges) are distributed outward from the origin. This default layout can be unintuitive for human analysis.



In practice, we shift the origin to the center of the frequency spectrum in the 2D DFT for visualization and interpretation purposes. Placing the zero frequency (DC component) in the middle makes it easier to identify and analyze low frequencies. These represent the most prominent or general features in the image, like overall brightness. High-frequency components, which correspond to fine details or noise, are placed outward from the center. This arrangement aligns better with how we interpret spatial and frequency information.

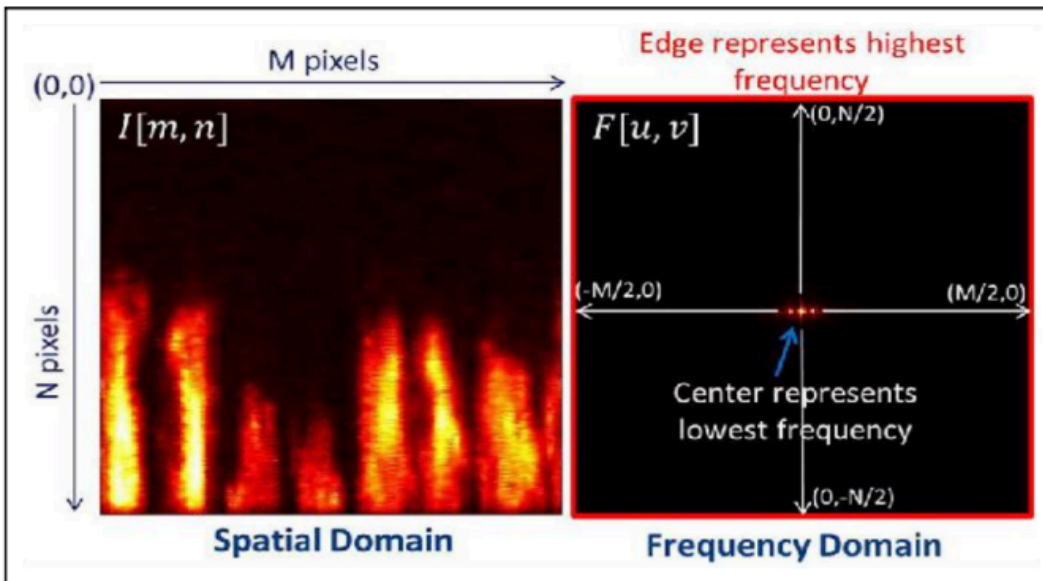


The Picture below clearly shows why shifting is crucial to make the frequency domain representation interpretable. The higher frequency contents in the horizontal axis due to variations along the horizontal direction are clearly visible in the shifted spectrum, but they were hard to visualize in the unshifted one.



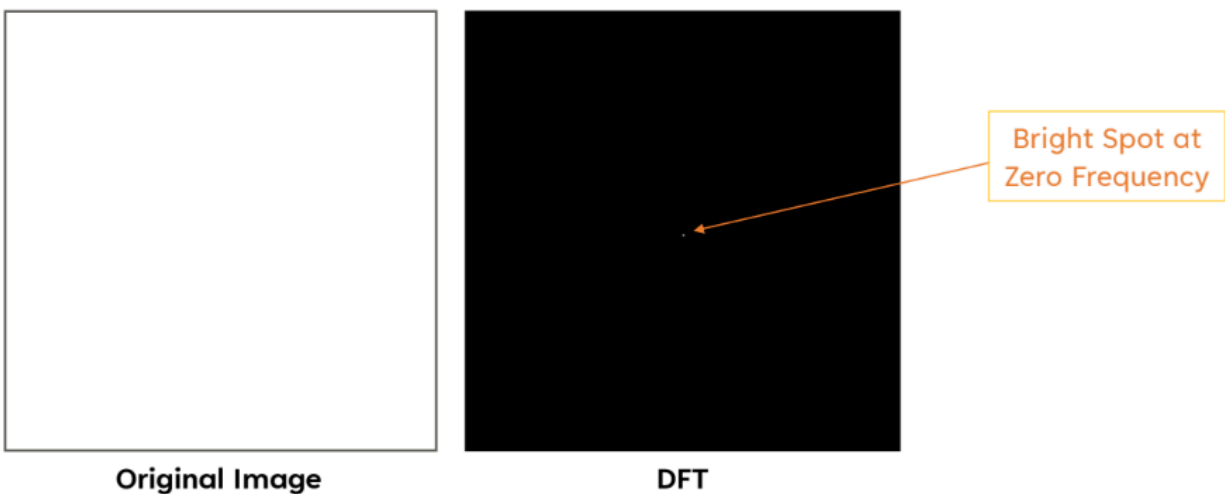
This image below illustrates the transition from the spatial domain to the frequency domain. On the left is the spatial representation of an image, I , where pixel values correspond to intensity variations. The right side shows its 2D frequency spectrum,

F after applying the DFT. The symmetric nature of the frequency spectrum is a property of real-valued spatial data.

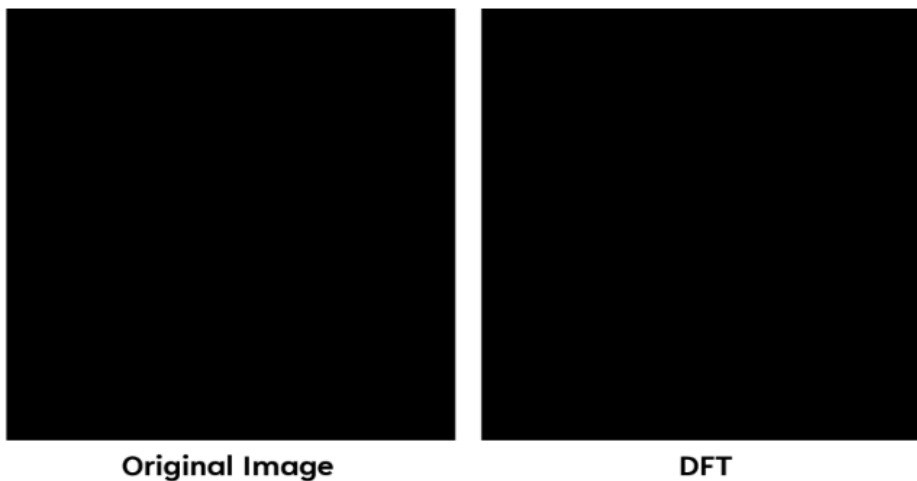


2D DFT: EXAMPLES

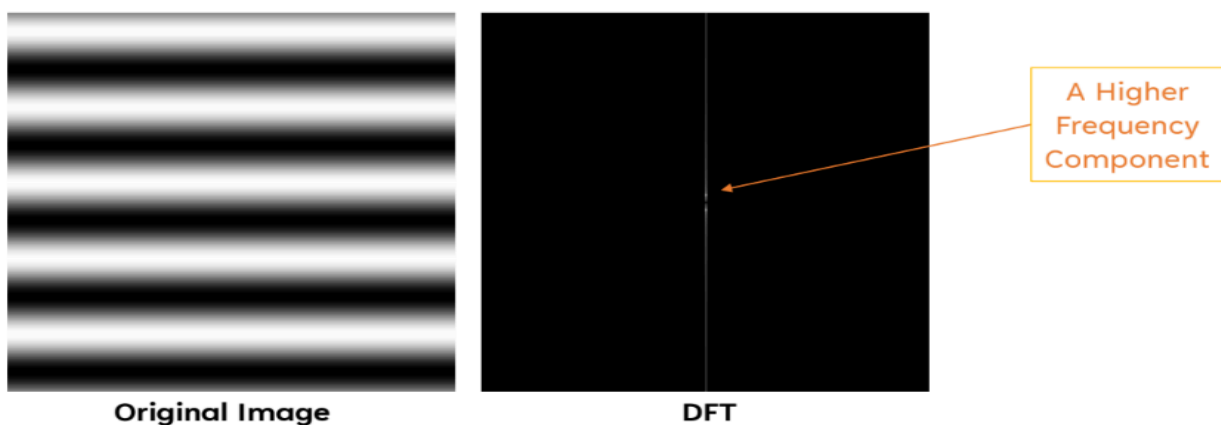
The DFT of a fully white image is a single bright spot at the center (DC component), indicating a uniform, zero-frequency intensity across the entire image. No other frequency component is visible, as there is no variation in the image at all. In a white image, all pixel values are 255, so there is an average value (255) and we got a bright spot.



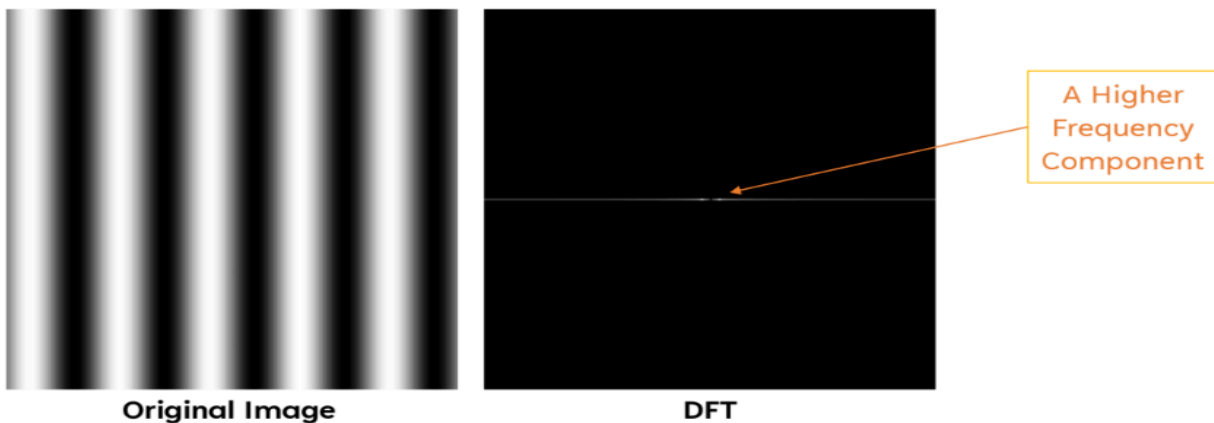
The DFT of a fully black image is completely dark because there is no signal (intensity is zero everywhere), and hence no frequency components. There is no value even at the center (average brightness is 0) and no frequency components as there are no variations in the image. In a dark image, all pixel values are 0, so there is no average and we got completely black DFT.



The DFT of this image shows high-frequency components along the vertical axis (y-direction), as the intensity varies vertically. As the variation follows a sinusoid pattern, there is a peak, a single bright higher frequency content (and its negative counterpart). The line that appears apart from the peaks arises due to boundary discontinuities in the image. If there is a sharp discontinuity between the edges of the image (boundaries), this creates artificial high-frequency components in the DFT (Spectral Leakage), manifesting as lines in the frequency spectrum, oriented perpendicularly to the direction of the variation (vertical in this case).



The DFT of this image shows high-frequency components along the horizontal axis (x-direction), as the intensity varies horizontally. As the variation follows the pattern of a sinusoid, there is a single bright higher frequency content (and its negative counterpart). Again, the spectral leakage that appears apart from the peaks arises due to boundary discontinuities in the image. The sharp discontinuity at the boundary of the image creates artificial high-frequency components in the DFT, manifesting as lines in the frequency spectrum.



2D DFT: REDUCING SPECTRAL LEAKAGE (OPTIONAL)

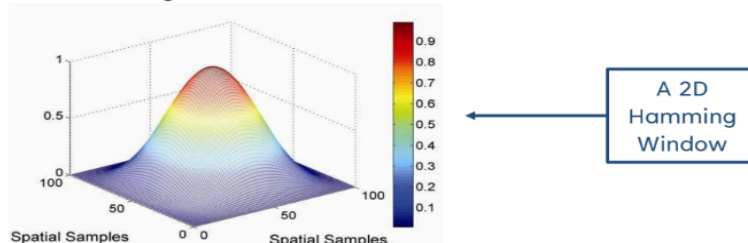
So, when analyzing a finite signal (e.g., images) in the frequency domain, the boundaries of the signal introduce discontinuities. These discontinuities result in spectral leakage, where energy from one frequency spreads into others. The Hamming Window reduces these discontinuities by tapering the signal's edges smoothly to zero (or nearly zero), rather than abruptly cutting it off.

The Hamming window is defined as:

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \leq n \leq N-1$$

$$w[m] = 0.54 - 0.46 \cos\left(\frac{2\pi m}{M-1}\right), \quad 0 \leq m \leq M-1$$

Where, M and N are the dimensions of the image and m and n are indices for rows and columns, respectively.



Hamming Window (in Images)

A **Hamming window** is used to **smooth the edges (boundaries)** of an image before applying transforms like the **Fourier Transform**.

What it Does

- It keeps the center pixels strong
- It gradually reduces pixel values toward the edges

So:

- Middle $\rightarrow$ weight ≈ 1 (**preserved**)
- Boundaries $\rightarrow$ weight $\rightarrow$ **small (close to 0, but not exactly 0)**

Why We Use It

When we process an image (especially in frequency domain), we usually assume:

- The image is **periodic (repeats itself)**

But in reality:

- Image edges **do NOT match** when repeated
- This creates **sharp discontinuities at boundaries**

Problem Without Windowing

- Abrupt change at edges $\rightarrow$ **sudden jump in intensity**
- Sudden jump = **high-frequency components**

Result:

- Unwanted noise in frequency domain
- Artifacts like ringing (Gibbs effect)

How Hamming Window Helps

- i. It **reduces boundary pixel values smoothly**
- ii. So edges fade gradually instead of cutting abruptly
- iii. This removes sudden jumps

Therefore:

- No sharp discontinuity
- Less **artificial high-frequency components**

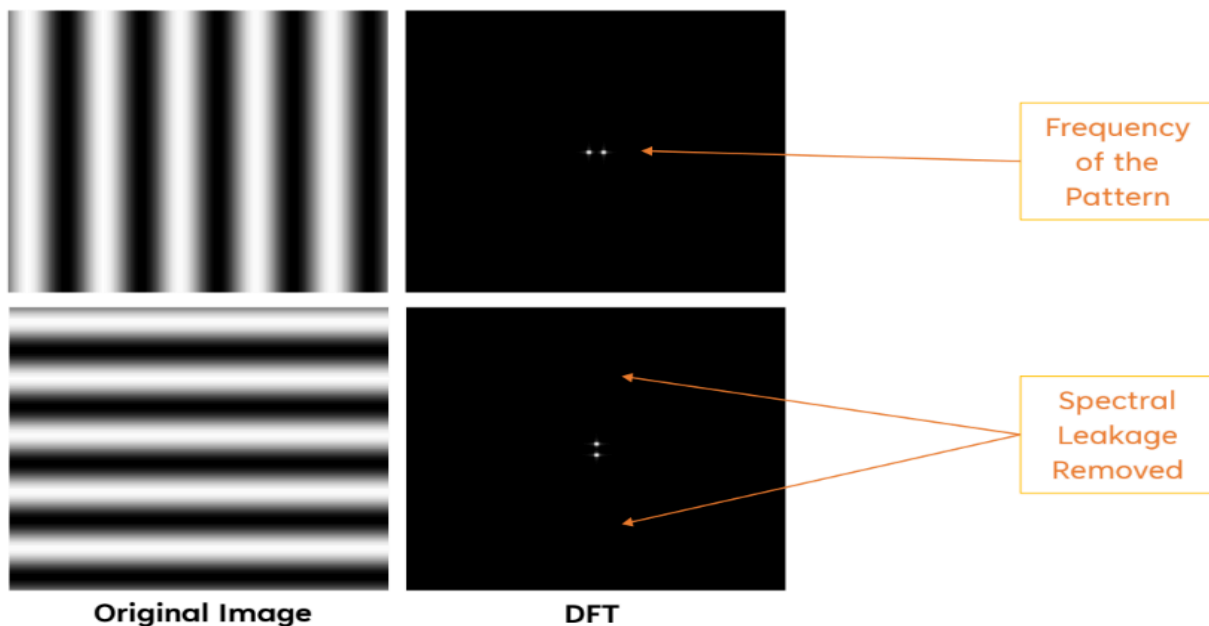
Intuition

Think of it like:

- Without window → image is **cut sharply**
- With Hamming window → image is **softly faded at edges**

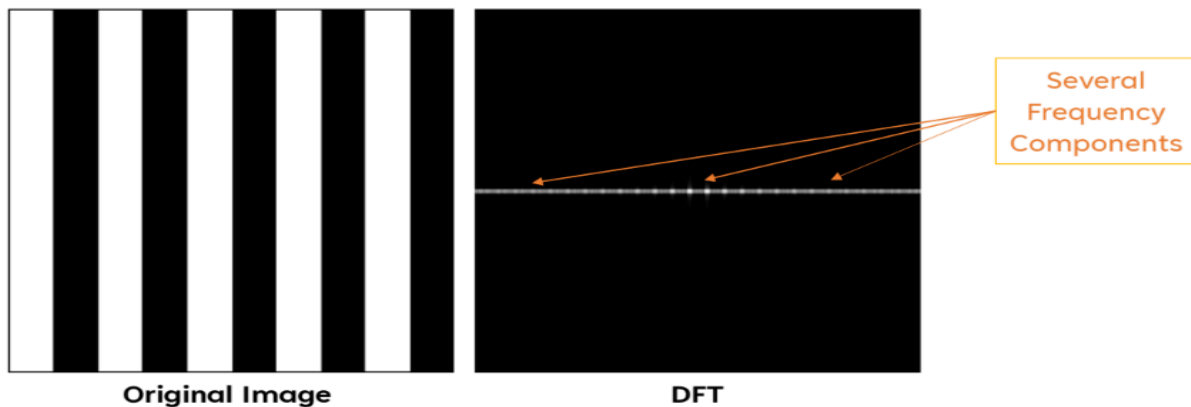
EFFECT OF HAMMING WINDOW

Look at the DFTs below after applying the hamming window. The frequencies of the pattern within the images are much clearer now, and spectral leakage has been reduced significantly.

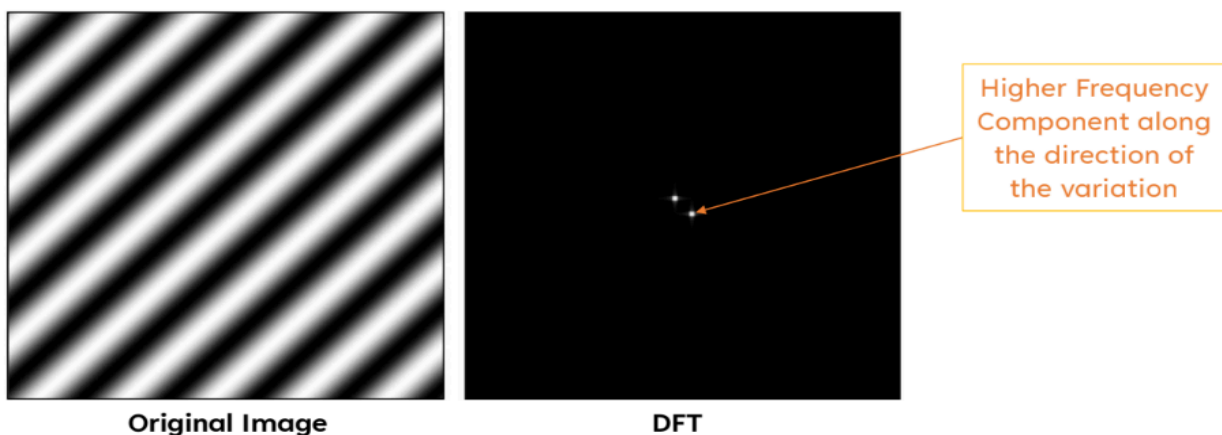


2D DFT: EXAMPLES

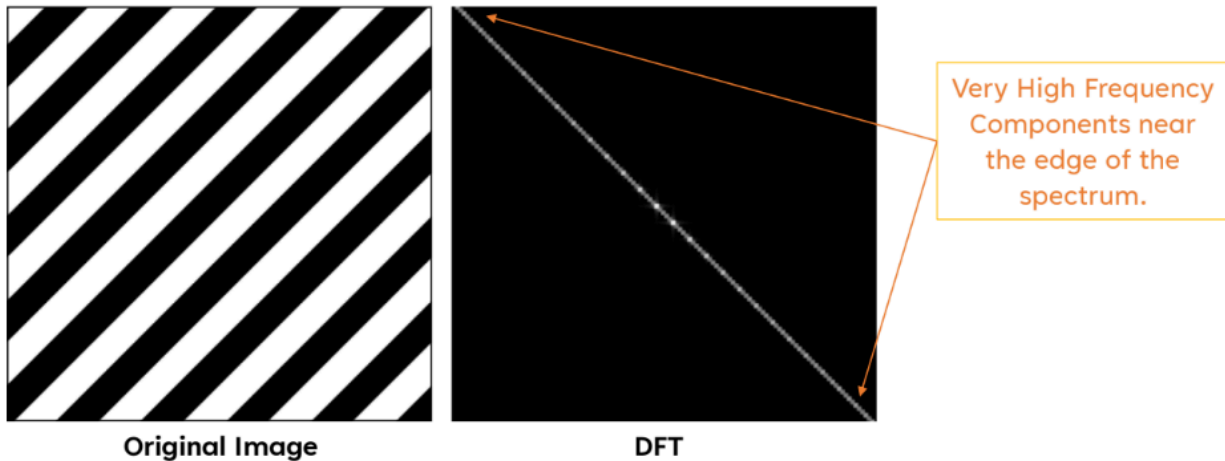
The DFT of this image shows high-frequency components along the horizontal axis (x-direction), as the intensity varies horizontally. As the image contains sharp changes in intensity (from fully black to white), there are several bright higher frequency contents (representing peaks along the vertical axis frequency spectrum) here. Sharp edges correspond to high frequencies, that is why we can see bright spots near the edge of the spectrum. There are no frequency contents bar zero along the vertical axis, due to no variations along the y-axis.



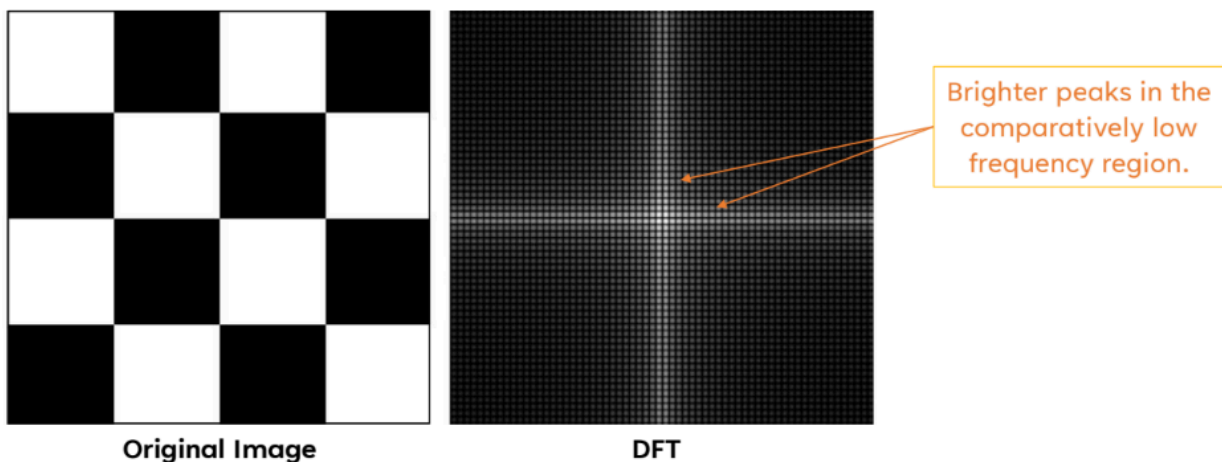
The DFT of this image displays a diagonal pattern of frequency components, corresponding to the direction and rate of intensity change along the angle. It contains a single peak (and its negative) along the angled direction of variation, corresponding to the frequency of the variation. As we can see, this frequency has both vertical and horizontal components, which is due to the fact that the intensity varies to some extent along both these directions.



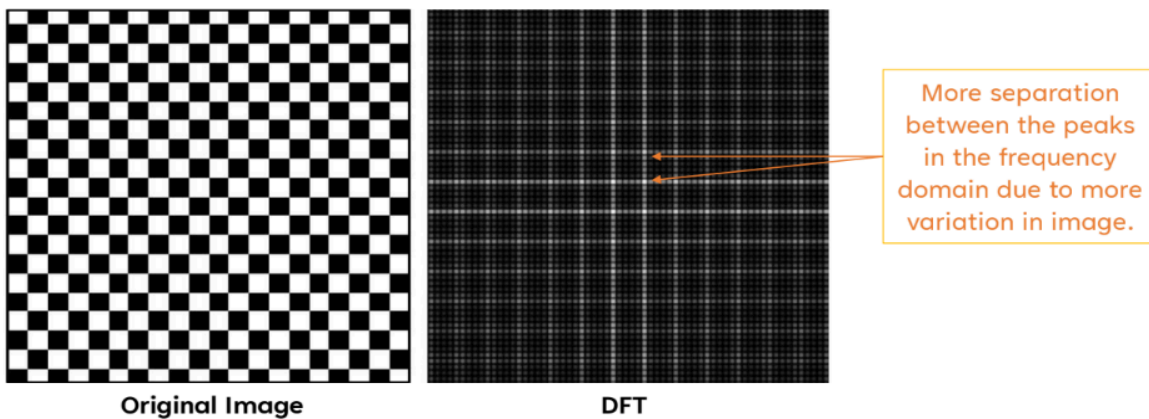
The DFT of this image shows sharp peaks along the diagonal axis, indicating strong periodic frequencies in the angled direction. As there are sharp edges along that angled direction, the DFT contains multiple peaks along the angled direction of variation, corresponding to the many frequencies the edges represent. Bright peaks near the corners of the spectrum show that edges contain quite strong high frequency components.



The DFT of this checkerboard contains symmetric peaks in both horizontal and vertical axes, representing periodic patterns in all directions. We can also notice that the peaks corresponding to comparatively lower frequencies are brighter, which is expected because the rate of variation in the checkerboard is not too much. Also, the adjacent peaks in the frequency domain are quite close to each other.

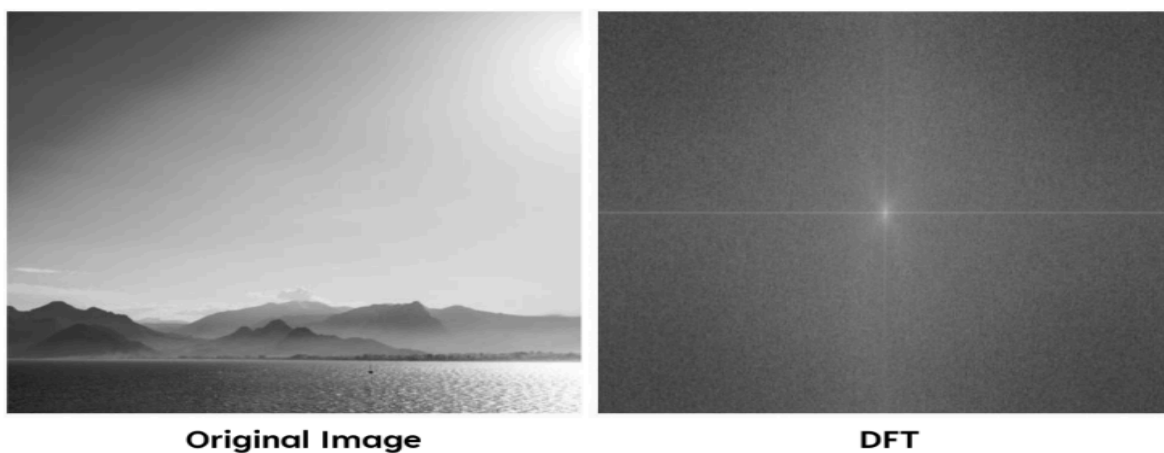


The DFT of this checkerboard again contains symmetric peaks in both horizontal and vertical axes, representing periodic patterns in all directions almost creating another checkerboard. The peaks corresponding to higher frequencies are now brighter, which is expected because the rate of variation in the checkerboard is much more in this case. Also, the distance between adjacent peaks in the frequency domain has increased due to the higher fundamental frequency and its harmonics (multiples of the fundamental frequency).



2D DFT: REAL LIFE IMAGES

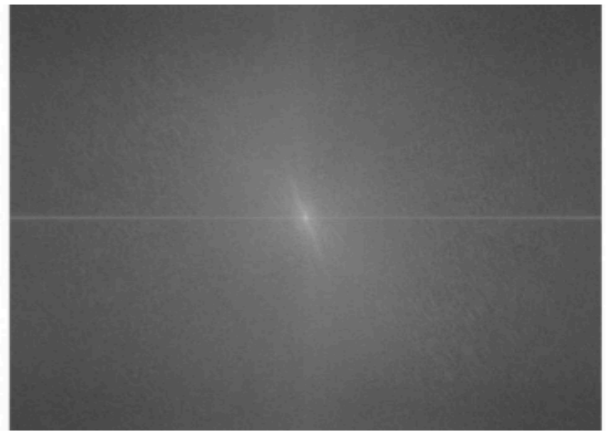
The DFT of this image below shows that the image mainly contains low frequency components, which is expected as there is little change in the skyline covering large portions of the picture. The peak is at a low frequency and there is a slightly bright region along the vertical axis, which is due the fact that the transition from sky to mountain to water happens in this direction.



The DFT of this image below once again shows that the image mainly contains low frequency components, which is obvious because there is hardly any variation throughout most of the picture. The peak is at a low frequency and there is a slightly bright region along the vertical axis at a small angle, which is again due the fact that the transition from sky to snowy mountain happens in this direction.



Original Image

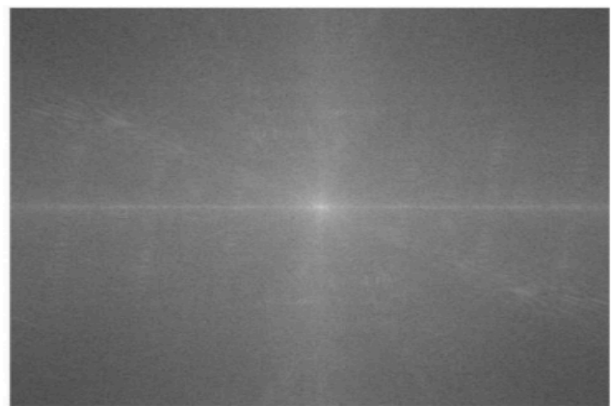


DFT

The DFT of this image below once again shows that the image contains both low & high frequency components. This is because the nighttime cityscapes with sharp lights from windows and neon signs show strong high frequencies. There are many bright regions throughout the frequency spectrum, even far from the center, which implies the existence of high-frequency components in the image.



Original Image



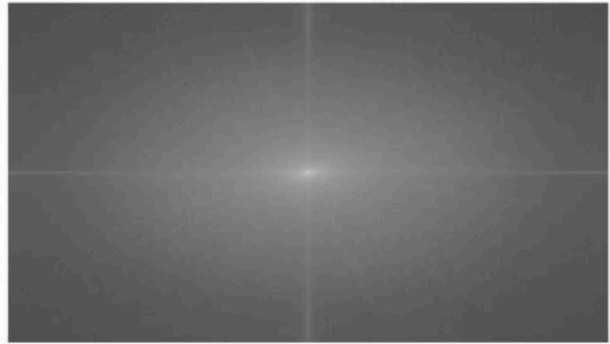
DFT

The DFT of another image containing both low & high frequency components. This is because this image also features lots of variations in intensity. There are

bright regions near the edge of the frequency spectrum, which implies the existence of high-frequency components in the image. Also notice how the dimensions of the DFT follows the dimensions of the original image (Why? Notice the 2D DFT formula!).



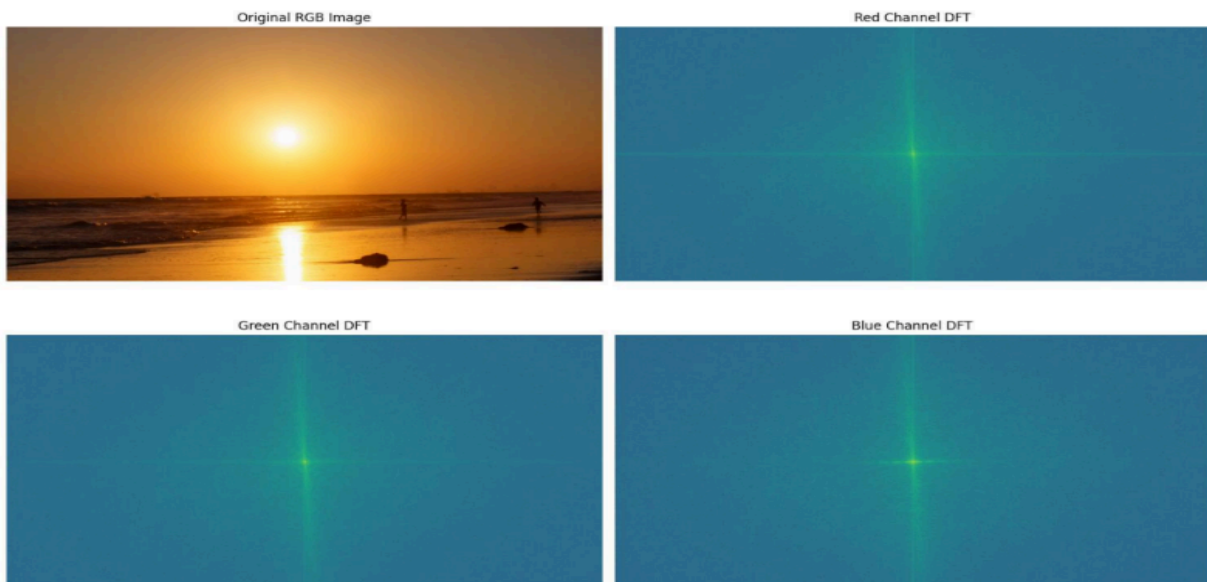
Original Image



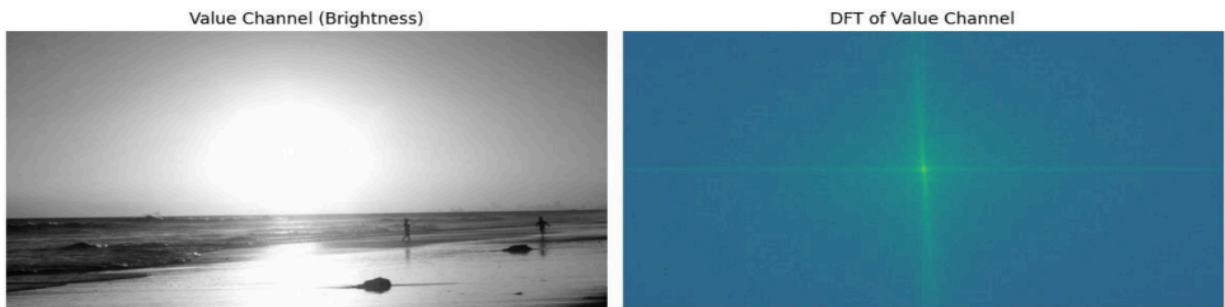
DFT

DFT OF RGB IMAGES

To compute the Discrete Fourier Transform (DFT) of an RGB image, one way could be applying the DFT to each of the 3 color channels (red, green, and blue) separately.



However, in RGB, the intensity and color information are mixed across all three channels. This makes it less straightforward to perform frequency-domain operations specific to brightness or color. That's why RGB images are often converted into alternate color spaces like HSV or LAB before applying a DFT. After conversion, operations like filtering or DFT are applied specifically to the channel specific to intensity (brightness), which is typically more important for frequency-domain analysis. For HSV, that channel is the Value channel which represents Brightness. Also, since color channels are untouched (Hue in HSV), the original colors remain intact even if intensity transforms are applied using the DFT.



2D INVERSE FOURIER TRANSFORM (IDFT)

The 2D Inverse Fourier Transform (IDFT) is the reverse process of the 2D Discrete Fourier Transform (DFT). While the DFT converts an image from the spatial domain (pixel values) into the frequency domain (frequency components), the IDFT converts it back from the frequency domain to the spatial domain. The 2D IDFT is vital for converting processed frequency data back into interpretable images in the spatial domain. Given the 2D DFT of an image, $F(u, v)$, of size $M \times N$, the 2D IDFT is given by:

$$f(x, y) = \frac{1}{M \cdot N} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

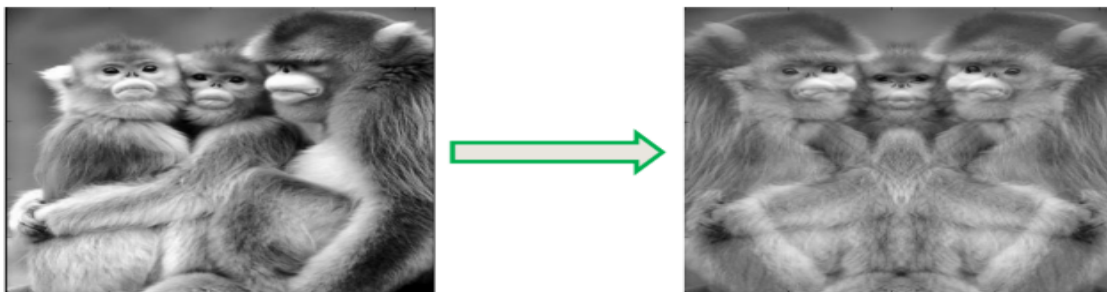
Where,

- $f(x, y)$ is the Intensity at spatial position x, y .

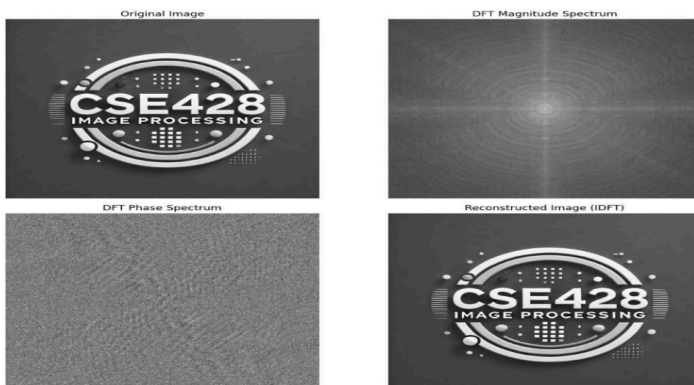
- $F(u, v)$ is the Frequency component at (u, v) .
- u, v are the horizontal and vertical frequency indices, respectively.
- M, N represent the dimensions of the image (Number of pixels along the x & y axis).

Clearly, like the value of 2D DFT $F(u, v)$, the value of 2D IDFT at each pixel location x, y is also a complex number. However, if the DFT was derived from a real valued signal (e.g., images), then the IDFT should approach a real value too, as the imaginary parts of the reconstructed spatial domain tend to be near zero, leaving the real part dominant.

It should be noted that even though when working with the DFT in image processing, we often work with the magnitude of the DFT and discard the phase information for visualization, both magnitude and phase are needed for the full reconstruction applying IDFT. Also, if the zero- frequency component was shifted to the center it needs to be shifted back to its original position. Otherwise, you may find out that your reconstructed image has been contaminated like this..



2D INVERSE FOURIER TRANSFORM (IDFT)



Example Problems

EXAMPLE 1

Compute the **2D Discrete Fourier Transform (DFT)** for the following **3X3** images. Present the results as matrices of complex numbers and provide the **magnitude spectrum** of the transformed images for visualization. **Comment** about the Frequency contents of these images. Can you relate them to any of the spatial image processing techniques?

Image 1

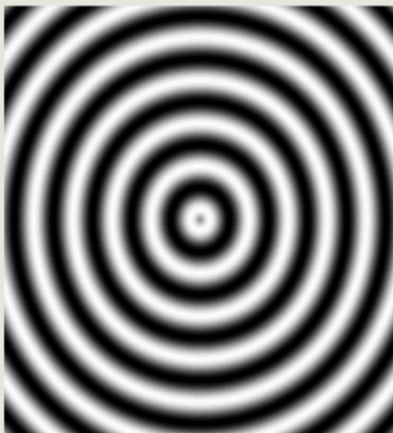
$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Image 2

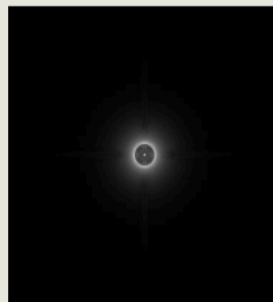
$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

EXAMPLE 2

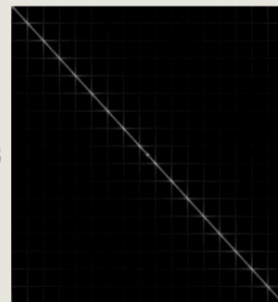
Observe the image below. The radial gradients follow sinusoidal pattern. Which DFT on the right do you think is the frequency domain representation of this image?



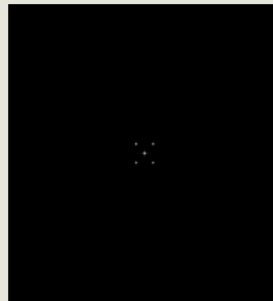
A



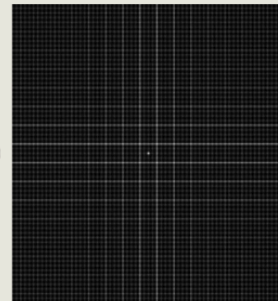
B



C

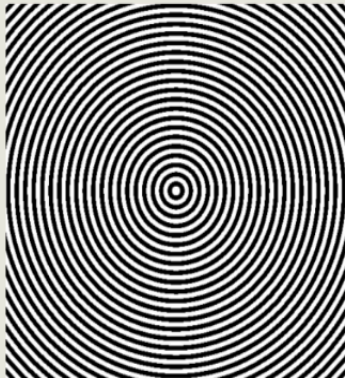


D

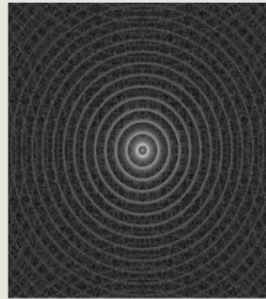


EXAMPLE 3

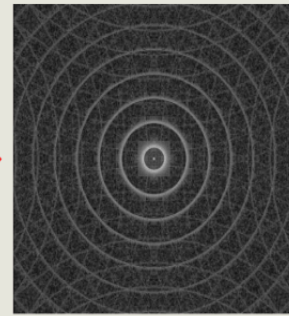
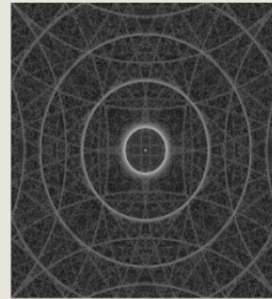
Observe the image **on the right** and its **DFT**. Which DFT below them (**A** or **B**) do you think is the frequency domain representation of the image **below this text**?



A



B

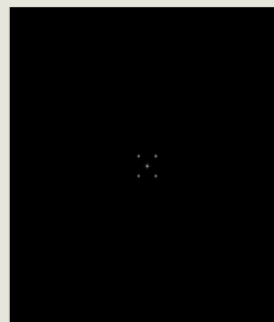


EXAMPLE 4

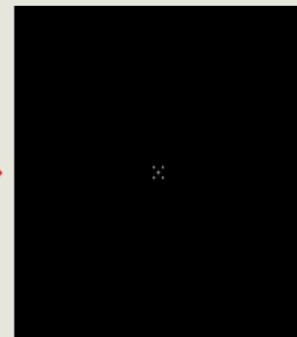
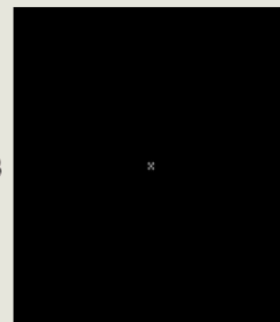
Observe the image **on the right** and its **DFT**. Which DFT below them (**A** or **B**) do you think is the frequency domain representation of the image **below this text**?



A



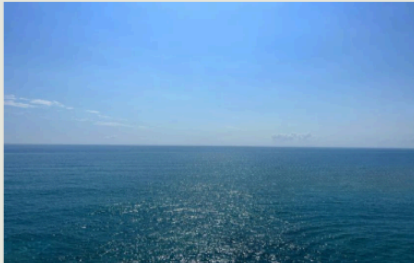
B



EXAMPLE 5

Observe the images **on the left**. Their DFTs are **on the right** column but we don't know which DFT corresponds to which image. Map them to their DFTs on the right using an arrow and explain your choices.

Images



DFTs

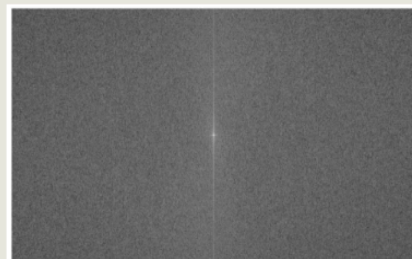
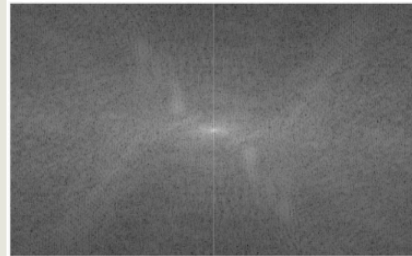


IMAGE PROCESSING IN THE FREQUENCY DOMAIN

Why Do We Use the Frequency Domain?

Even though image processing **can be done in the spatial domain**, transforming an image into the **frequency domain** gives several powerful advantages.

1. Efficient Convolution & Filtering

- In the **spatial domain**:
 - Convolution (filtering) = **heavy computation**, especially for large kernels
- In the **frequency domain**:
 - Convolution becomes **simple multiplication**

Convolution $\leftrightarrow$ Multiplication

Result:

- Faster processing
- More efficient for large filters (blur, sharpening, etc.)

2. Easy Separation of Image Information

The frequency domain gives a **global view** of the image.

- **Low frequencies**:
 - Smooth regions
 - Background, lighting
- **High frequencies**:
 - Edges
 - Fine details
 - Noise

This makes it easy to:

- Remove noise (cut high frequency)

- Smooth image (keep low frequency)
- Enhance edges (boost high frequency)

3. Advanced Processing Techniques

Many important applications rely on frequency domain:

Image Compression

- Example: JPEG
- Removes less important frequencies → reduces file size

Noise Reduction

- Remove unwanted frequency components selectively

Feature / Pattern Recognition

- Frequency representation captures **structure and patterns**
- Useful in:
 - Texture analysis
 - Object detection

4. Stability to Some Transformations (Refined Idea)

- The **magnitude spectrum** is:
 - Less sensitive to **small spatial shifts**
 - Useful for recognizing patterns

Small correction:

- It is **not fully invariant** to scaling and rotation
- But can still help in analyzing such variations

5. Key Intuition

Spatial domain:

- “Where” things are (pixel values)

Frequency domain:

- “How” things change (patterns, edges, textures)

CONVOLUTION VS MULTIPLICATION

1. Convolution in Spatial Domain

- Convolution means:
 - **Sliding a filter (kernel)** over the image
 - At each position → multiply + sum overlapping pixels

Operation:

- Many **repeated calculations**
- Overlapping regions → **computationally expensive**, especially for large filters

2. Why It Becomes Complex

- For an image of size ($M \times N$) and a kernel of size ($K \times K$):
 - Each pixel requires (K^2) operations
- Large kernel → huge computation

That’s why spatial convolution is **slow for large filters**

3. Convolution Theorem (Core Idea)

The most important concept:

Convolution in Spatial Domain $\Leftrightarrow$ Multiplication in Frequency Domain

and

Multiplication in Spatial Domain $\Leftrightarrow$ Convolution in Frequency Domain

4. What Happens in Frequency Domain

Steps:

1. Convert image $\rightarrow$ frequency domain (using DFT)
2. Convert filter $\rightarrow$ frequency domain
3. Do **element-wise multiplication**
4. Convert back (IDFT)

No sliding, no overlapping - just direct multiplication

5. Why Multiplication is Better

- No repeated overlap calculations
- Faster for large images and filters
- Efficient with algorithms like **FFT**

So:

- Spatial domain $\rightarrow$ intuitive but slow
- Frequency domain $\rightarrow$ efficient and faster

6. Intuition

Think of it like:

- Spatial convolution = **manual mixing step-by-step**
- Frequency multiplication = **mix everything at once**

Convolution in the spatial domain is computationally expensive due to repeated overlapping operations.

Using the **Convolution Theorem**, we convert it into simple element-wise multiplication in the frequency domain, making processing much more efficient.

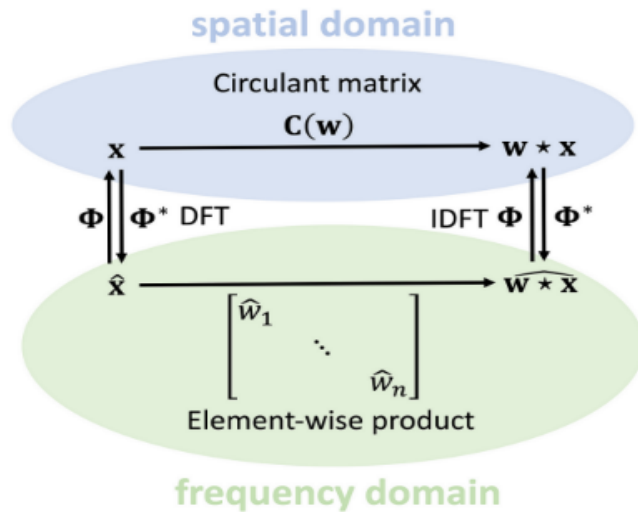
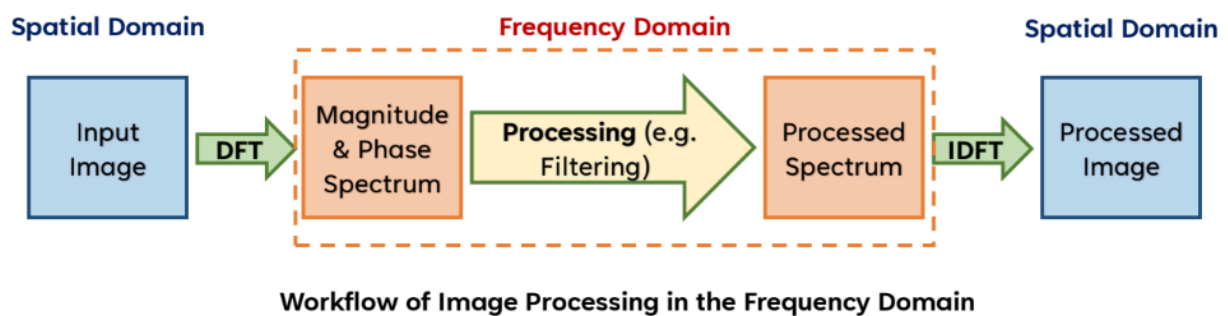


IMAGE PROCESSING WORKFLOW

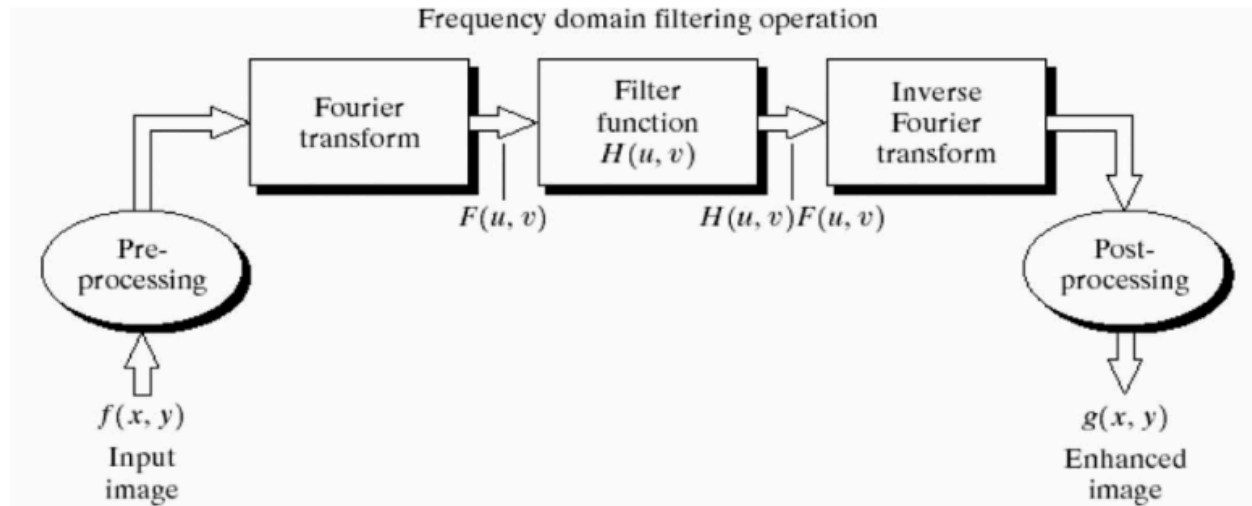
Unlike the spatial domain, the frequency-domain image processing workflow starts by using the Fourier Transform or similar methods to transform the image into its frequency-domain representation. In this transformed space, image processing operations like filtering, noise reduction, or enhancement take place. Once the processing is done, the Inverse Fourier Transform brings the image back into the interpretable spatial domain. The final output is a processed image that's been improved or cleaned up using the advantages of frequency-domain processing.



FILTERING IN FREQUENCY DOMAIN

Filtering in the frequency domain involves manipulating the frequency components of an image to achieve specific effects like smoothing, sharpening, or noise removal. This process is different from filtering in the spatial domain in the sense that the

filter is applied directly to the frequency representation of the image, often as a multiplication operation with a frequency-domain mask, instead of the more complex convolution operation we saw in the spatial domain. After filtering, the Inverse Fourier Transform is used to convert the modified frequency data back to the spatial domain.



TYPES OF FILTERS

Low-Pass Filter (LPF)

Keeps smooth parts, removes sharp changes.

Example:

- Blurring a noisy image to remove grain/noise
- Removing salt-and-pepper noise from a photo

High-Pass Filter (HPF)

Keeps edges, removes smooth background.

Example:

- Edge detection in an image (detect object boundaries)
- Sharpening a slightly blurred image

Band-Pass Filter (BPF)

Keeps only a specific frequency range.

Example:

- Extracting texture patterns (like fabric details)
- Keeping medium details while removing both noise and background

Band-Stop Filter (Notch Filter)

Removes specific unwanted frequencies.

Example:

- Removing periodic noise (striped interference in an image)
- Eliminating repeating patterns caused by sensor errors

The Filtering Operation for images in the frequency domain is simply multiplying the filter with the input signal:

Filtered Spectrum, $G(u, v) = F(u, v) \bullet H(u, v)$

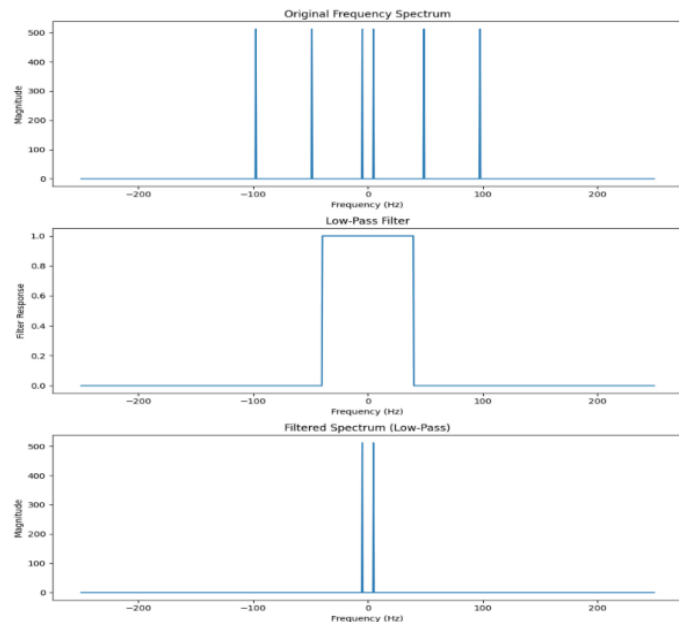
Where, $F(u, v)$ is the frequency domain representation of the original signal and $H(u, v)$ is the frequency domain filter.

FILTERING IN 1D: LOW PASS

Observe how a **low pass** filter works in 1D.

The filter is designed with a cutoff frequency (**40** in this case), which defines the threshold between the frequencies it passes and those it blocks. Frequencies below this cutoff are retained, while those above are suppressed.

The filter **multiplies the Fourier-transformed signal by a filter mask**. For frequencies below the cutoff, the mask values are **1** (full pass), and for frequencies above, the mask values are **0** (completely blocked) or gradually decrease (if a smoother transition is needed). After multiplication with 0, the high frequency components also become 0 and get canceled.

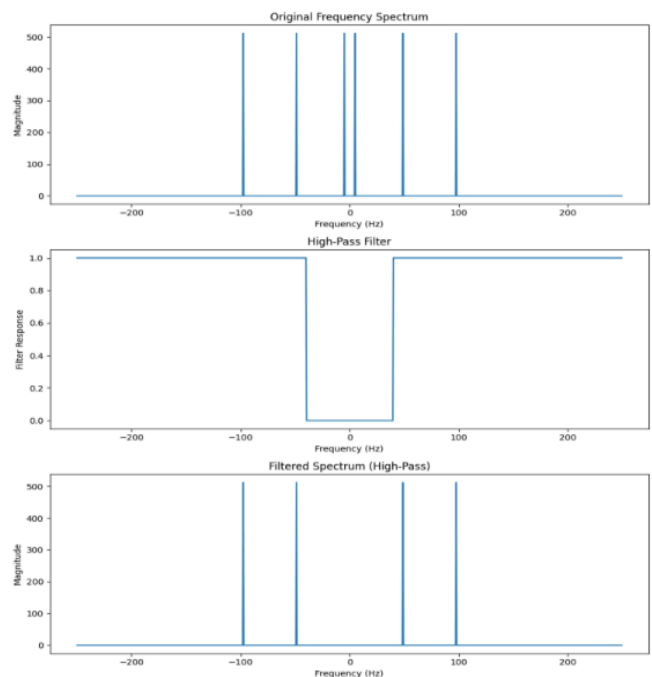


FILTERING IN 1D: HIGH PASS

Observe how a **high pass** filter works in 1D.

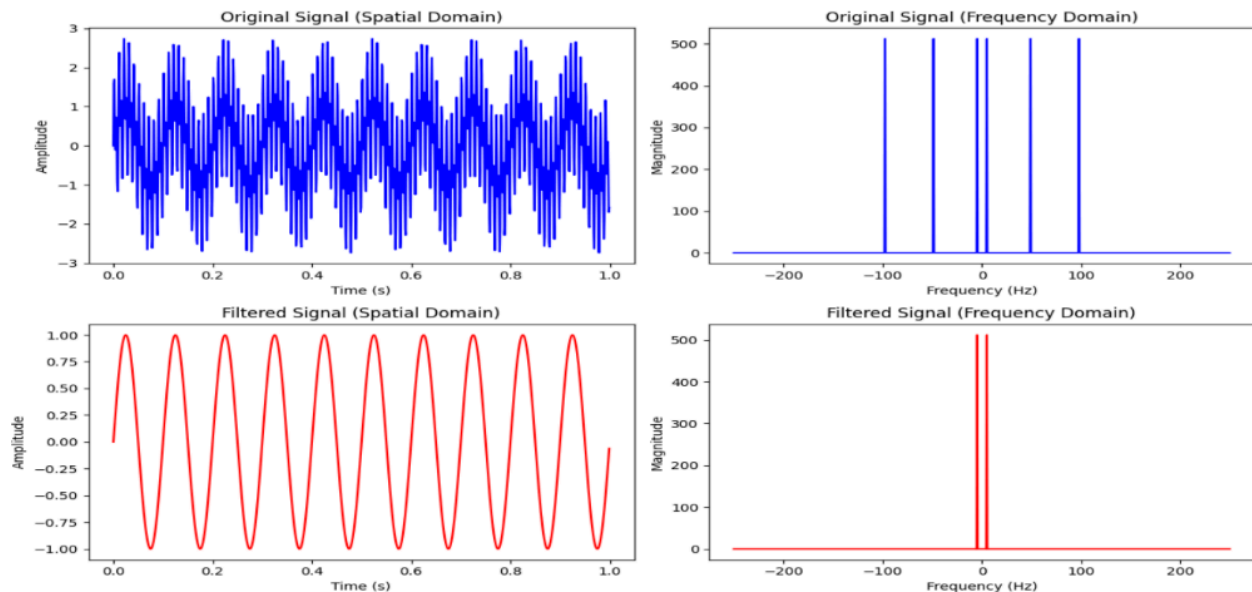
The filter is designed with a cutoff frequency (**40** in this case), which defines the threshold between the frequencies it passes and those it blocks. Frequencies below this cutoff are suppressed, while those above are retained.

The filter too **multiplies the Fourier-transformed signal by a filter mask**. For frequencies above the cutoff, the mask values are **1** (full pass), and for frequencies below, the mask values are **0** (completely blocked) or gradually decrease (if a smoother transition is needed). After multiplication with 0, the low frequency components also become 0 and get cancelled.



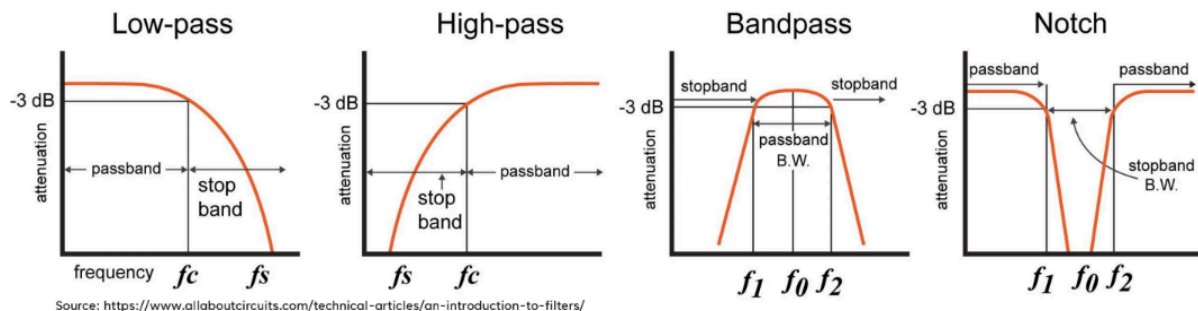
FILTERING IN 1D: EFFECT IN SPATIAL DOMAIN

Observe how the 1D signal has been smoothed by eliminating rapid variations after applying the low pass filter in the frequency domain.



DIFFERENT TYPES OF FILTERS IN 1D

Take a look at different types of 1D filters below. The negative frequencies have been ignored as they convey the same information as the positive frequencies.



Source: <https://www.allaboutcircuits.com/technical-articles/an-introduction-to-filters/>

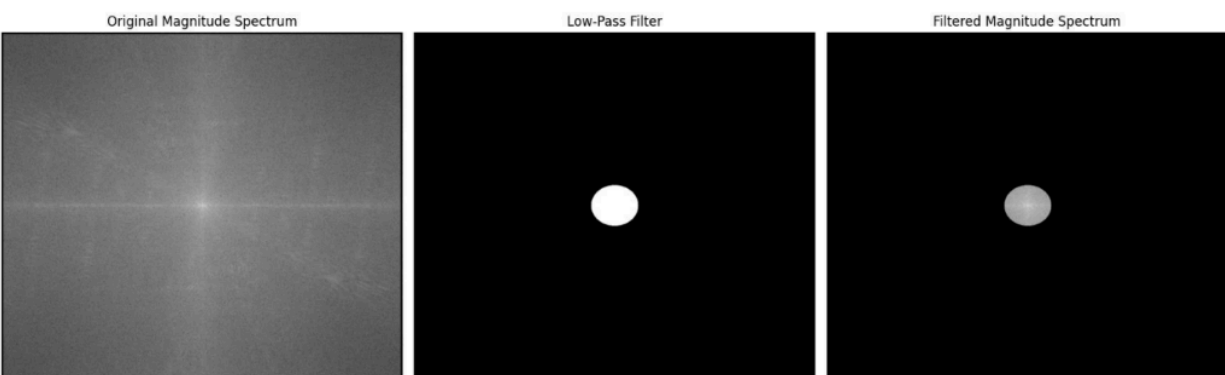
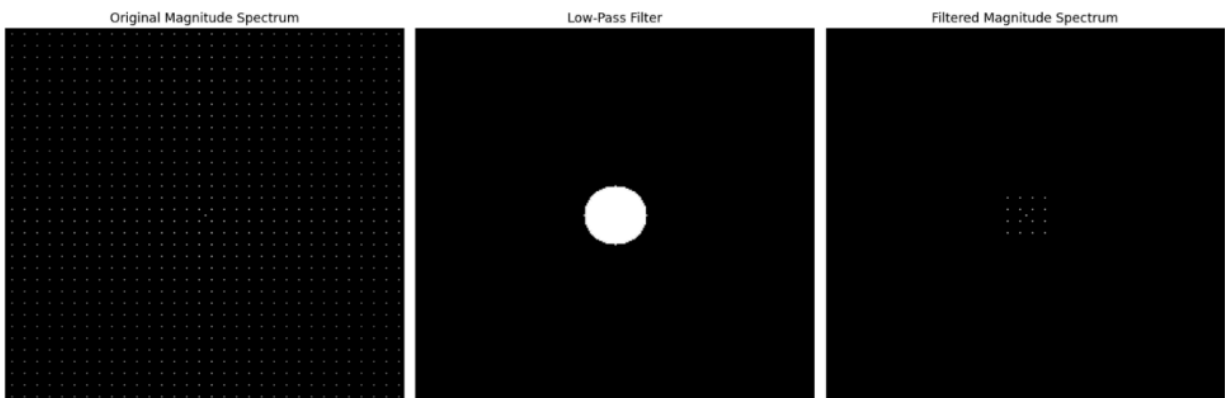
Frequency Domain Filters in 1D

FILTERING IN 2D IMAGES: LOW PASS

Observe how a low-pass filter works in 2D images.

The filter is also designed with a cutoff frequency, which defines the threshold between the frequencies it passes and those it blocks. Frequencies below this cutoff are retained, while those above are suppressed.

For frequencies below the cutoff, the mask values are 1 (full pass), and for frequencies above, the values are 0 (completely blocked) or gradually decrease (if a smoother transition is needed). After multiplication with 0, the high-frequency components also become 0 and get canceled.

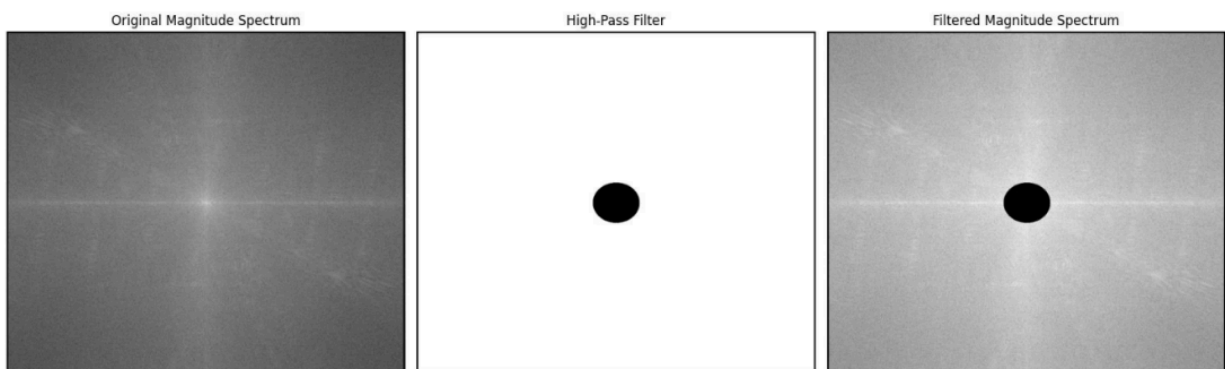
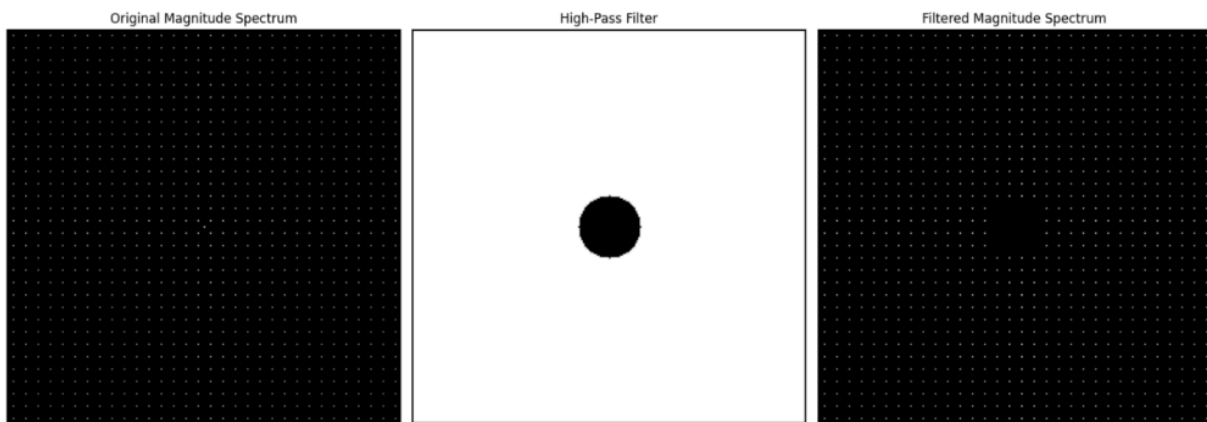


FILTERING IN 2D IMAGES: HIGH PASS

Observe how a high-pass filter works in 2D images.

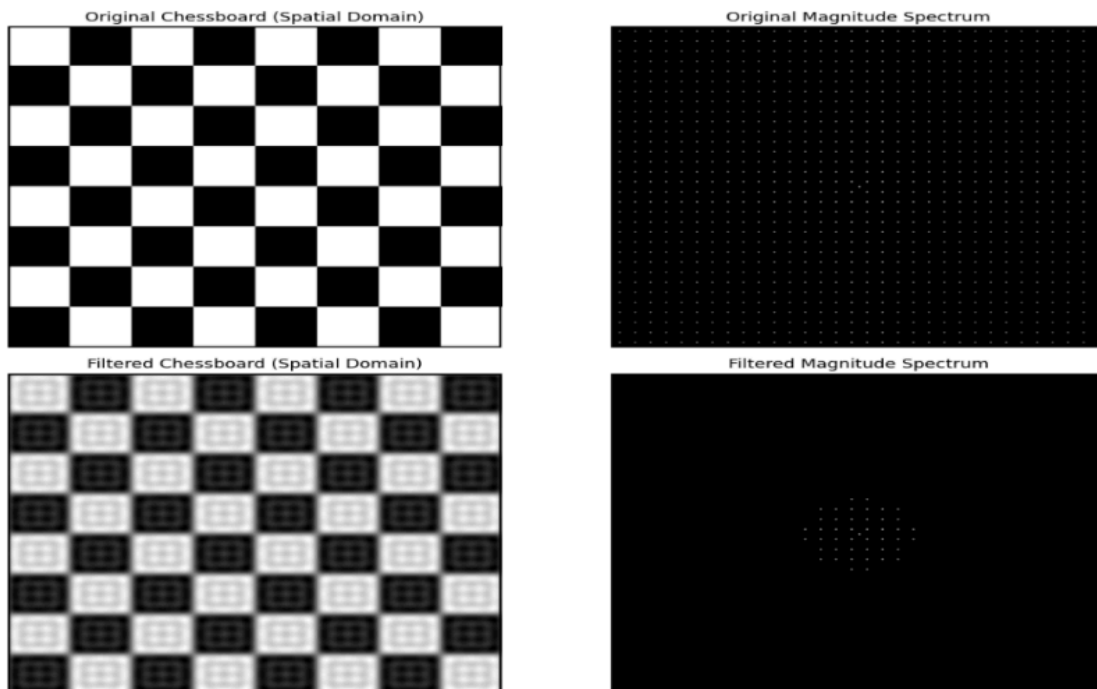
The filter is once again designed with a cutoff frequency, which defines the threshold between the frequencies it passes and those it blocks. Frequencies above this cutoff are retained, while those below are suppressed.

For frequencies above the cutoff, the mask values are 1 (full pass), and for frequencies below, the values are 0 (completely blocked) or gradually decrease (if a smoother transition is needed). After multiplication with 0, the low-frequency components also become 0 and get canceled.



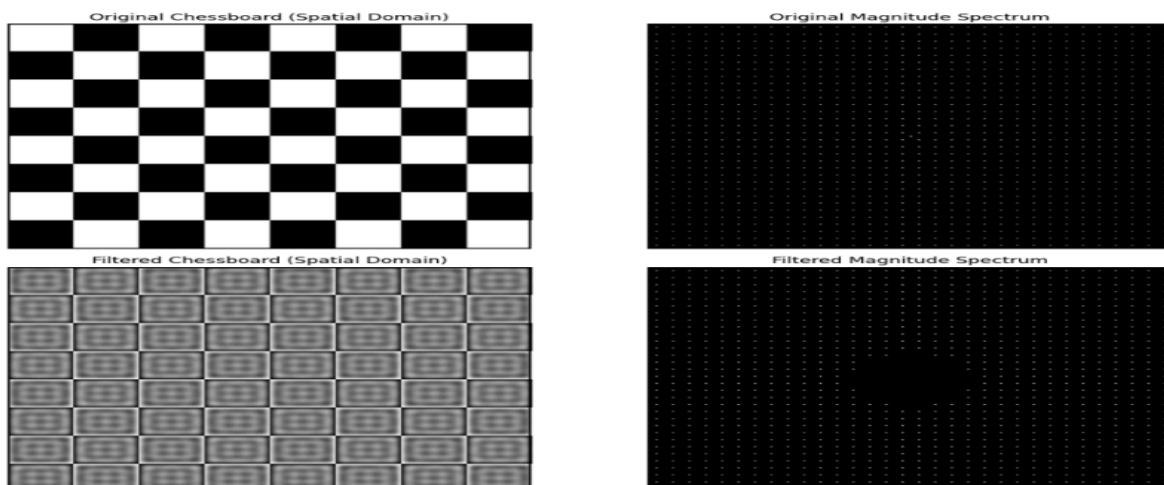
FILTERING IN 2D: LOW PASS

After applying the low pass filter in the frequency domain, observe how the 2D signal has been blurred by smoothing out sharp edges.

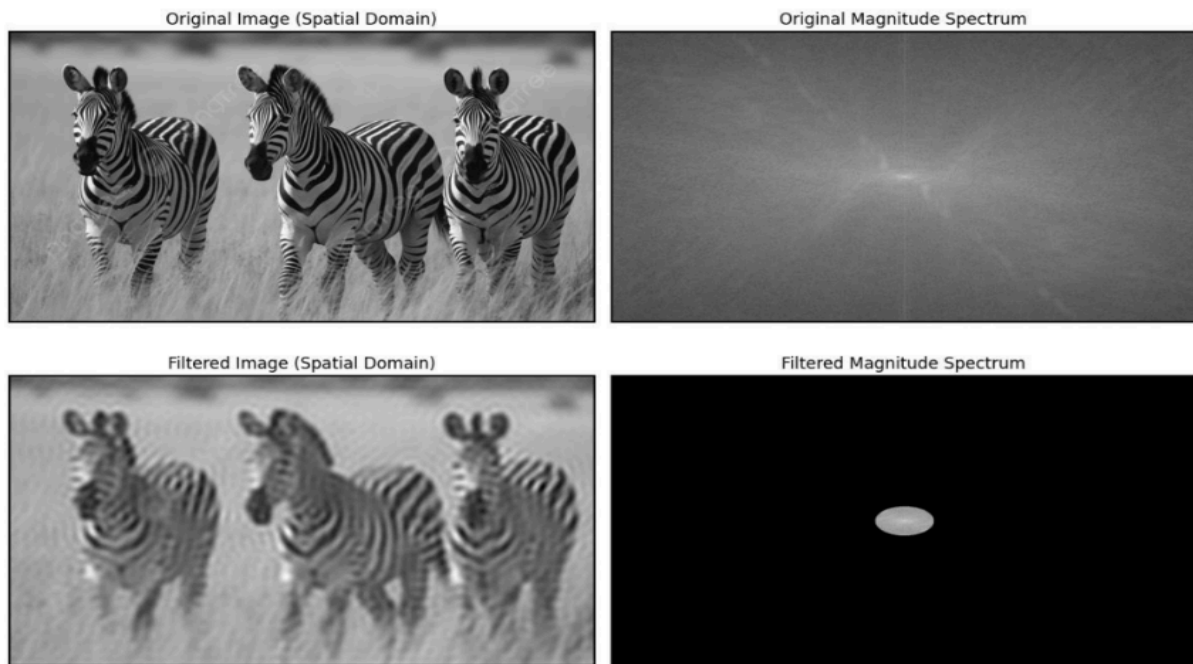


FILTERING IN 2D: HIGH PASS

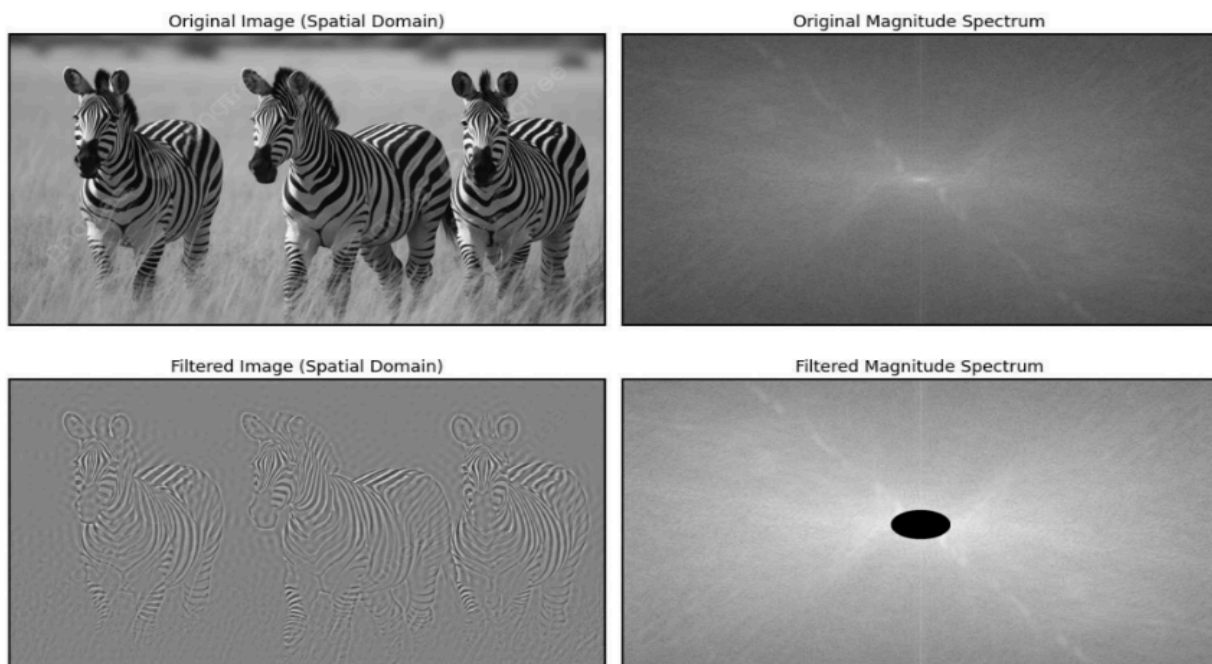
After applying the high pass filter in the frequency domain, observe how the 2D signal has been sharpened by enhancing sharp edges and canceling out the smoother regions.



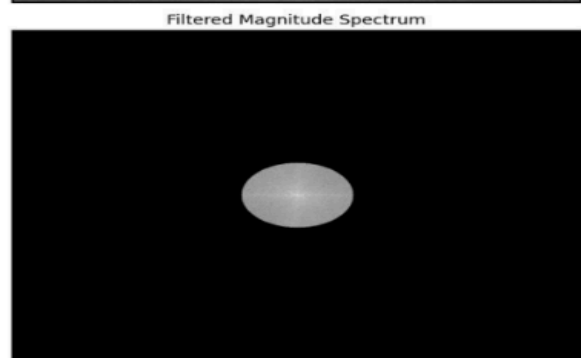
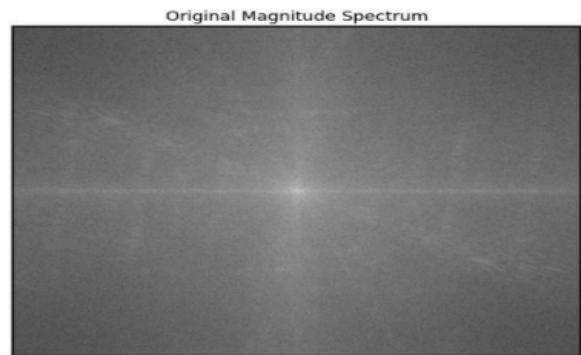
FILTERING IN 2D: LOW PASS



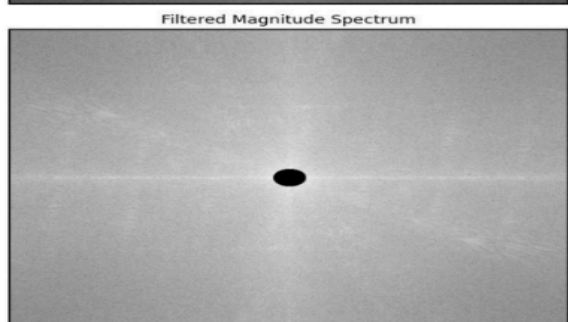
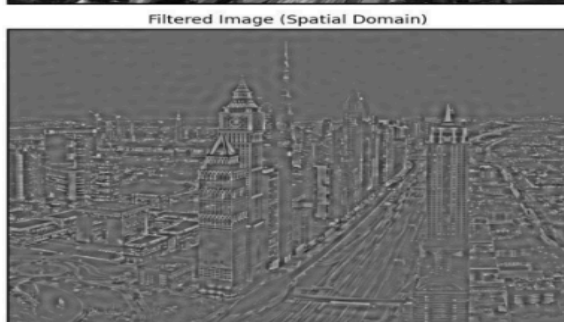
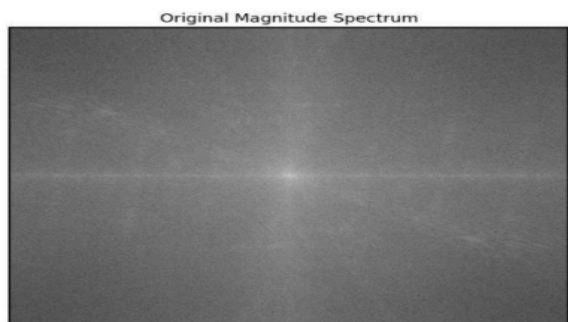
FILTERING IN 2D: HIGH PASS



FILTERING IN 2D: LOW PASS



FILTERING IN 2D: HIGH PASS

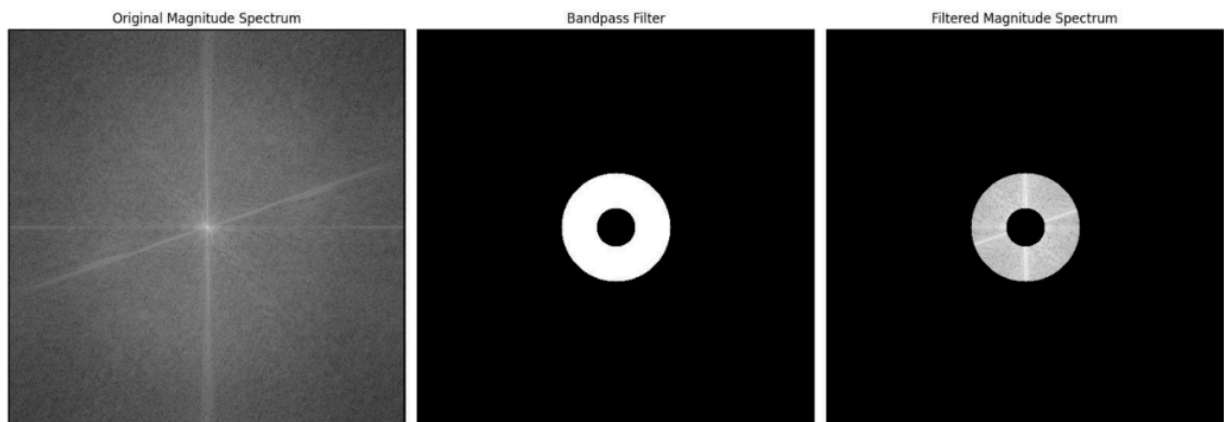


FILTERING IN 2D IMAGES: BAND PASS

Observe how a band-pass filter works in 2D images.

The filter is designed with two cutoff frequencies, which define the thresholds between the frequencies it passes and those it blocks. Frequencies above the higher cutoff and below the lower cutoff are suppressed, while those between these two cutoffs are retained.

For frequencies between the cutoffs, the mask values are 1 (full pass), and for frequencies above and below both, the values are 0 (completely blocked) or gradually decrease (if a smoother transition is needed).

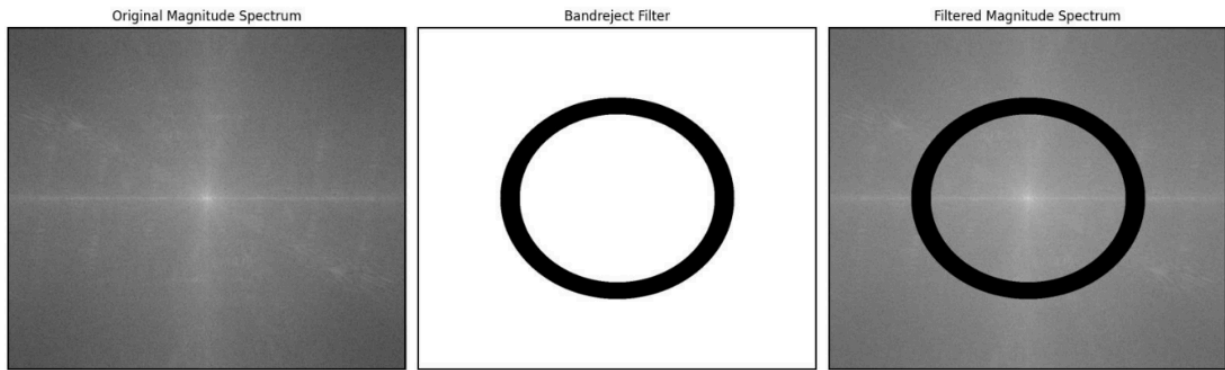


FILTERING IN 2D IMAGES: BAND STOP (NOTCH)

Observe how a band-stop (Notch) filter works in 2D images.

The filter is also designed with two cutoff frequencies, which define the thresholds between the frequencies it passes and those it blocks. Frequencies above the higher cutoff and below the lower cutoff are retained, while those between these two cutoffs are suppressed.

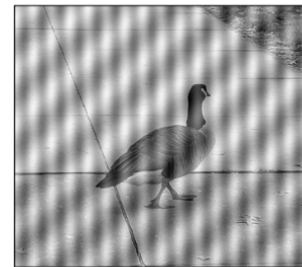
For frequencies above and below the cutoffs, the mask values are 1 (full pass), and for frequencies between them, the values are 0 (completely blocked) or gradually decrease (if a smoother transition is needed).



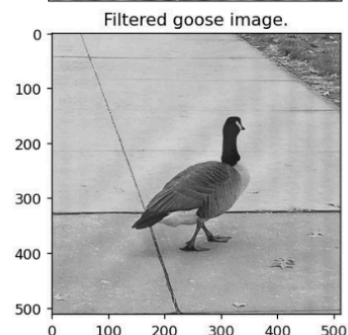
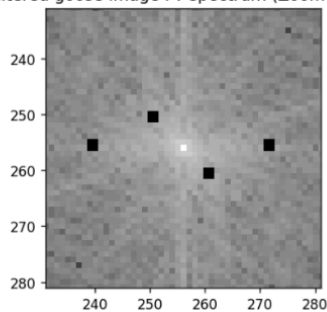
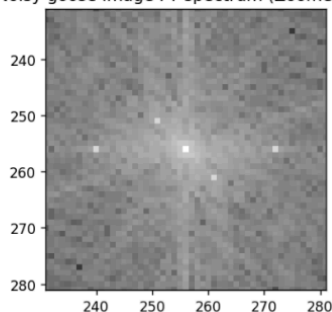
FILTERING IN 2D IMAGES: BAND STOP (NOTCH)

Observe how a **band-stop (Notch)** filter has been applied to eliminate noisy patterns containing specific frequencies to produce a clean image.

The DFT of the image on the right shows specific frequencies that represent the unwanted patterns in the horizontal and an angled direction. Then, filtering was applied to reject only those specific frequencies to get the filtered denoised image.



Noisy goose image FT spectrum (Zoomed in). Filtered goose image FT spectrum (Zoomed in).



Gaussian Filter

- Weights follow a **bell-shaped curve**
- **Center pixel** → **highest weight**
- **Far pixels** → **lower weight**

So nearby pixels influence more than distant ones

Why It's Useful

- Produces **smooth, natural blurring**

- Preserves **gradual intensity changes** (edges look softer, not destroyed)
- Mimics **human visual perception**

Example:

- Slightly noisy image → apply Gaussian filter
- Result:
 - Noise reduced
 - Edges remain smooth (not too sharp, not too blurry)

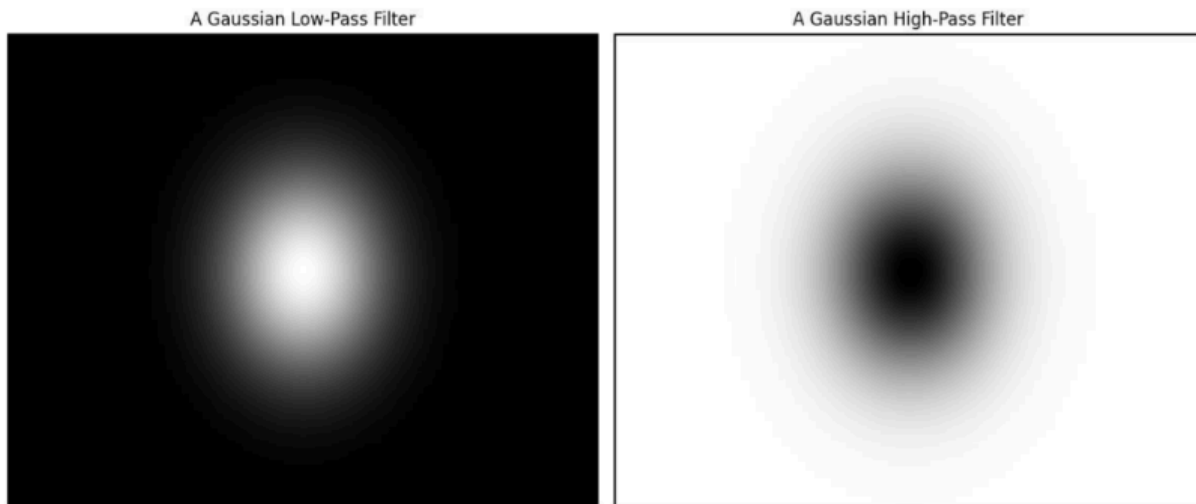
Better than simple averaging filter (which can over-blur)

Frequency Domain Insight

- Gaussian in spatial domain $\Leftrightarrow$ Gaussian in frequency domain
- It **smoothly reduces high frequencies** (no abrupt cutoff)

Result:

- No ringing artifacts
- Very natural filtering



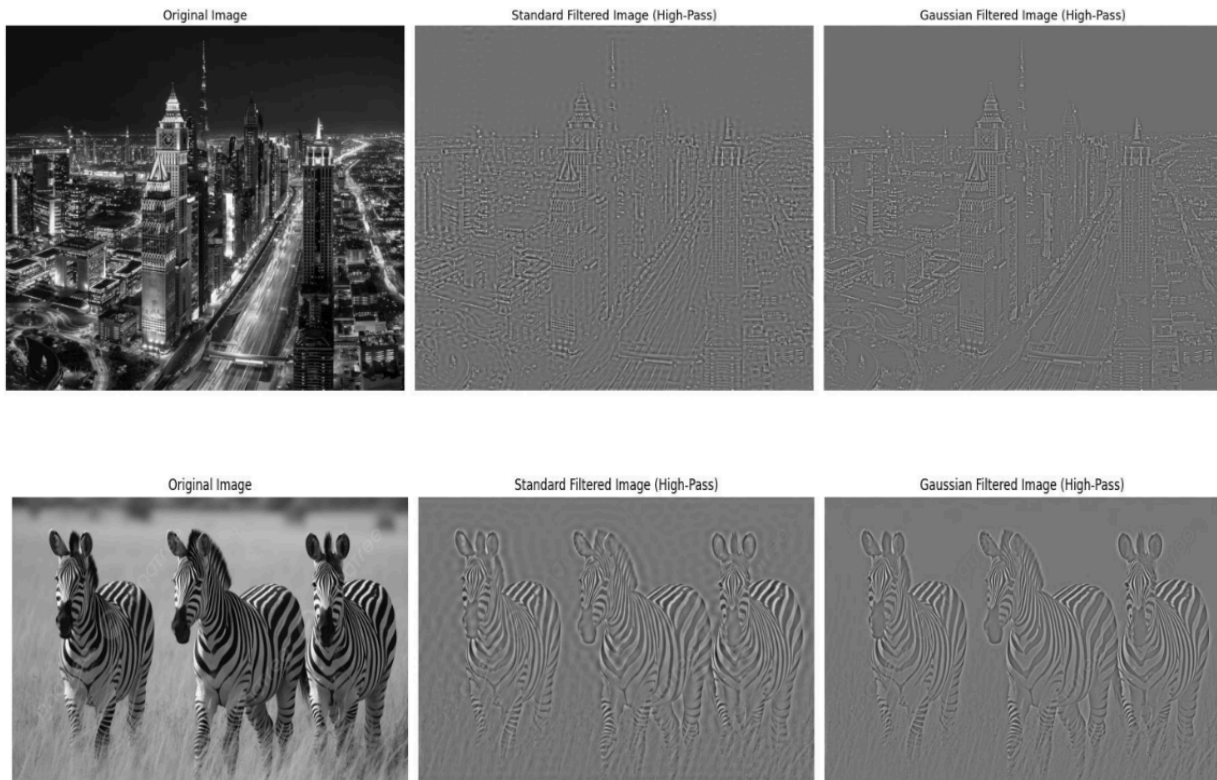
GAUSSIAN FILTERS: NOISE REDUCTION

Observe how Gaussian Low Pass Filters perform better at removing salt & pepper noise while preserving the picture quality compared to the Standard Low Pass Filters with similar cut-off frequencies.



GAUSSIAN FILTERS: EDGE DETECTION

Observe how a Gaussian High Pass Filter enhances the edges and reveals the textures/structures much more efficiently compared to a Standard High Pass Filter with a similar cut-off frequency.



Filtering in Color Images

- Directly filtering **RGB channels separately** can cause:
Color distortion
Unnatural results

So we convert to another color space where:

- **Brightness (intensity)** is separate
- **Color information** is separate

Common Color Spaces

- **HSV** $\rightarrow$ (Hue, Saturation, Value)
- **YCbCr** $\rightarrow$ (Y = luminance, Cb/Cr = chrominance)

How Filtering is Done

1. Convert RGB $\rightarrow$ HSV or YCbCr
2. Apply filter **only on intensity**:
 - HSV $\rightarrow$ **Value (V)**
 - YCbCr $\rightarrow$ **Y (luminance)**
3. Keep color components unchanged
4. Convert back $\rightarrow$ RGB

Why This Works

- Only **brightness/structure** is modified
- Colors remain **unchanged and natural**

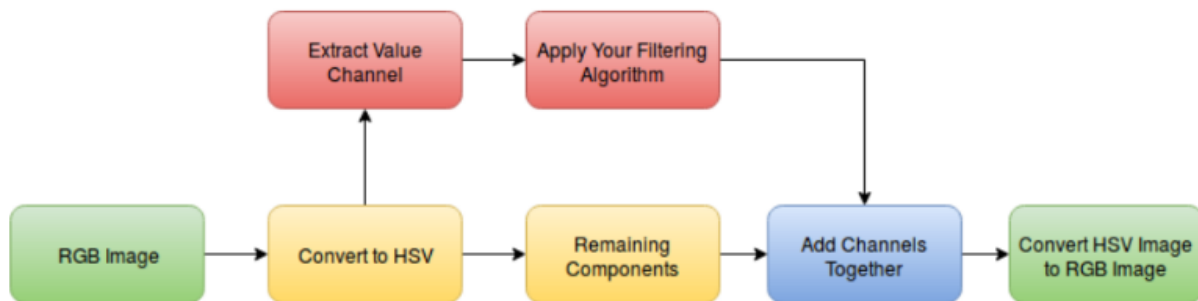
Example

Example: Noise Reduction

- Noisy color image
- Convert to **YCbCr**
- Apply Gaussian filter on **Y channel only**

Result:

- Noise reduced
- Colors stay accurate (no weird color mixing)



Filtering by Converting RGB Images to HSV Color Space

FILTERING IN COLOR IMAGES: LOW PASS

Observe the effect of applying low-pass filtering on the Value channel in the HSV color space and adding the filtered output to the untouched remaining channels (Saturation and Hue) to convert them back to RGB in the images below.



Original Image



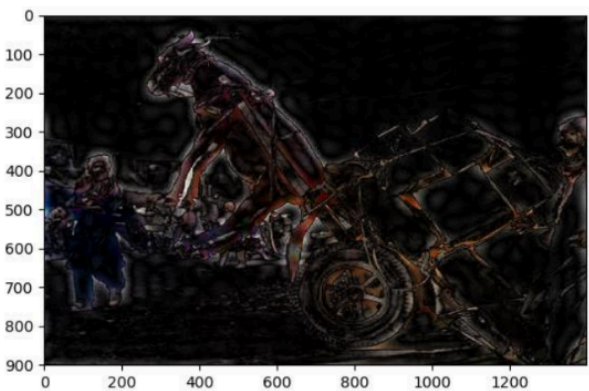
Filtered Image
(Low-Pass)

FILTERING IN COLOR IMAGES: HIGH PASS

Observe the effect of applying high-pass filtering on the Value channel in the HSV color space and adding the filtered output to the untouched remaining channels (Saturation and Hue) to convert them back to RGB in the images below.



Original Image



Filtered Image
(High-Pass)

High-Boost Filtering (Idea)

- Enhances **high-frequency components** (edges, details)
- Still keeps **low-frequency content** (overall image)

Result: **Sharper image without losing brightness**

Formula

$$H_{hb}(u,v) = A \cdot H_{lp}(u,v) + (1 - H_{lp}(u,v))$$

Meaning of Terms

- $(H_{\{hb\}}(u,v)) \rightarrow$ High-boost filter
- $(H_{\{lp\}}(u,v)) \rightarrow$ Low-pass filter
- $(A > 1) \rightarrow$ Boosting factor

Intuition

- $(H_{\{lp\}}) \rightarrow$ keeps smooth part (original image)
- $((1 - H_{\{lp\}})) \rightarrow$ extracts **high-frequency details**

So:

- First term $\rightarrow$ keeps original image (scaled by A)
- Second term $\rightarrow$ adds edges/details

Role of Boosting Factor (A)

- $(A = 1) \rightarrow$ normal high-pass filtering
- $(A > 1) \rightarrow$ **high-boost filtering**

Larger (A):

- More emphasis on original image
- Stronger sharpening effect

Example

- Slightly blurred image
- Apply high-boost filter with ($A = 1.5$)

Result:

- Edges become sharper
- Image does NOT become too dark (low frequencies preserved)

Original Image



High-Boost Filtered ($A=0.5$)



Original Image



High-Boost Filtered ($A=0.1$)



Original Image

High-Boost Filtered ($A=1.5$)

Original Image

High-Boost Filtered ($A=-0.5$)